

Object-Oriented Programming with C++

What We Will Cover

1. Introduction to OOP
2. C++ Programming Concepts
3. Functions in C++
4. Classes and Objects
5. Constructor and Destructor
6. Operator Overloading
7. Virtual Functions & Polymorphism
8. Inheritance
9. Template and Namespace
10. Exception handling
11. File Handling

CHAPTER 1

Introduction to Object-Oriented Programming

Procedural vs OOP · Objects · Classes · Inheritance · Reusability · Polymorphism · Applications

Procedural vs Object-Oriented Programming

- Procedural: program = sequence of functions; data is separate and often global
- OOP: program = collaborating objects, each bundling its own data + behaviour
- Procedural: top-down design | OOP: bottom-up, class-based design
- OOP maps naturally onto real-world entities: Car, Account, Employee
- Examples — Procedural: C, FORTRAN | OOP: C++, Java, Python

Real-life analogy: *Procedural = an open warehouse anyone can walk into. OOP = vending machines — you interact only through their buttons (methods).*

Core Characteristics of OOP

- **Object:** a concrete instance with its own state + behaviour (a specific car)
- **Class:** the blueprint that defines what objects of that type will have (the car's design)
- **Inheritance:** a derived class acquires and extends a base class ('is-a')
- **Reusability:** code written once (in a class) is reused via inheritance/composition
- **Polymorphism & Overloading:** same interface, different behaviour depending on type/arguments

Applications of OOP

- Graphical user interfaces (windows, buttons, menus)
- Simulation & modelling (banking, traffic, games)
- CAD/CAM and object-oriented databases
- Client-server & distributed systems
- Embedded / real-time systems, robotics
- Large-scale enterprise software (ERP, banking systems)

CHAPTER 1 — PROGRAM 1.1

A tiny OOP model: BankAccount

```
class BankAccount {
private:
    string owner; double balance;    // state
public:
    BankAccount(string n, double b) { owner=n; balance=b; }
    void deposit(double amt) { balance += amt; }    // behaviour
    void showBalance() {
        cout << owner << "'s balance: " << balance << endl;
    }
};

int main() {
    BankAccount acc1("Sita", 5000), acc2("Hari", 12000);
    acc1.deposit(1500); acc2.deposit(500);
    acc1.showBalance(); acc2.showBalance();
}
```

Output: Sita's balance: 6500 | Hari's balance: 12500

Practice & Exam-style Questions

1. Differentiate procedural vs OOP with an example.
2. Explain any four OOP characteristics with real-life examples.
3. List 5 application areas of OOP and justify each.
4. Differentiate class and object with a program.



Probable Exam Question

Q1: What is OOP? Explain any four features with examples. Differentiate OOP from POP. [PU Exam 2022]

Q2: "C++ is a hybrid language." Justify this statement.

CHAPTER 2

C++ Programming Concepts

cin/cout · Operators · Type conversion · Arrays & Pointers · new/delete · Manipulators · const · enum

CHAPTER 2

Introduction to C++

- Developed by Bjarne Stroustrup at Bell Labs, starting in 1979.
- Originally named "C with Classes" before being renamed C++.
- Designed as an extension of the C programming language.
- Adds object-oriented programming (OOP) features while retaining C's low-level control and performance.
- Every C++ program must have one `main()` function, which serves as the program's entry point.
- Core building blocks include:
 - ✓ Headers (`#include`)
 - ✓ Namespaces (e.g., `using namespace std;`)
 - ✓ Statements
 - ✓ Functions
- C++ is a compiled language.
- It is **Statically typed**.
 - ***"Statically typed"** means that the **data type of every variable must be known at the time the program is compiled, before the program actually runs.***
- It supports multiple programming paradigms: procedural, object-oriented, and generic programming.

CHAPTER 2

Introduction to C++

Real-life Analogy

Learning C++ is like learning to **cook in a professional, fully-equipped kitchen** rather than with just a campfire (C). You get all of C's raw control over your ingredients (memory, pointers) plus organized stations, standardized tools, and recipes (classes, the Standard Library) that let you build far more complex dishes reliably.

```
#include <iostream>    // header for input/output
using namespace std;  // avoids writing std:: everywhere

int main() {          // program entry point
    cout << "Hello, Object Oriented World!" << endl;
    return 0;         // 0 signals successful execution to the OS
}
```

Explanation

`#include <iostream>` brings in the input/output stream library. `using namespace std;` lets us write `cout` instead of `std::cout`. Execution always begins at `main()`. `cout << "..."` prints to the console, and `return 0;` tells the operating system the program finished without errors.

CHAPTER 2

Streams: Extraction (>>) and Insertion (<<)

- The **extraction operator** >> (Input): Reads (extracts) data **from** an input stream (like cin, the keyboard) **into** a variable.

```
cin >> a >> b;
```

- The **insertion operator** << (stream insertion / output operator) : sends (inserts) data from a variable or literal into an output stream (like cout, the screen).

```
cout << "City: " << city;
```

- Both are ordinary C++ operators overloaded by the library for streams

```
#include <iostream>    // header for input/output
using namespace std;   // avoids writing std:: everywhere

int main() {           // program entry point
    string name;
    int age;

    cout << "Enter your name and age: ";
    cin >> name >> age;    // extracts two values from the input stream

    cout << "Hello " << name << ", you are " << age << " years old." << endl;
    return 0;
}
```

CHAPTER 2

Reference Variables

- A **reference variable** is an alias (alternative name) for an existing variable.
- It is declared using the **ampersand (&)** symbol.
- Once a reference is initialized to a variable, it **cannot** be changed to refer to another variable.
- References are **not separate variables** – they directly refer to the memory location of the original variable.

Syntax:

```
int x = 10;  
int &ref = x;    // ref is a reference to x
```

CHAPTER 2

Scope Resolution Operator

The **scope resolution operator (::)** is used to access global variables, class members, and namespaces.

It allows you to specify the scope in which an identifier belongs.

It is especially useful when a **local variable hides a global variable** or when defining member functions outside the class.

Key Uses

- **Access global variables** when shadowed by local variables.
- **Define member functions** outside the class definition.
- **Access static members** of a class.
- **Access namespaces** and their members.
- **Access class constants** and enumerations.

Syntax:

```
scope_name :: identifier
```

Here,

- **scope** – It can be a class name, namespace, or, global.
- **identifier** – The identifier can be a variable, function, type, or, constant.

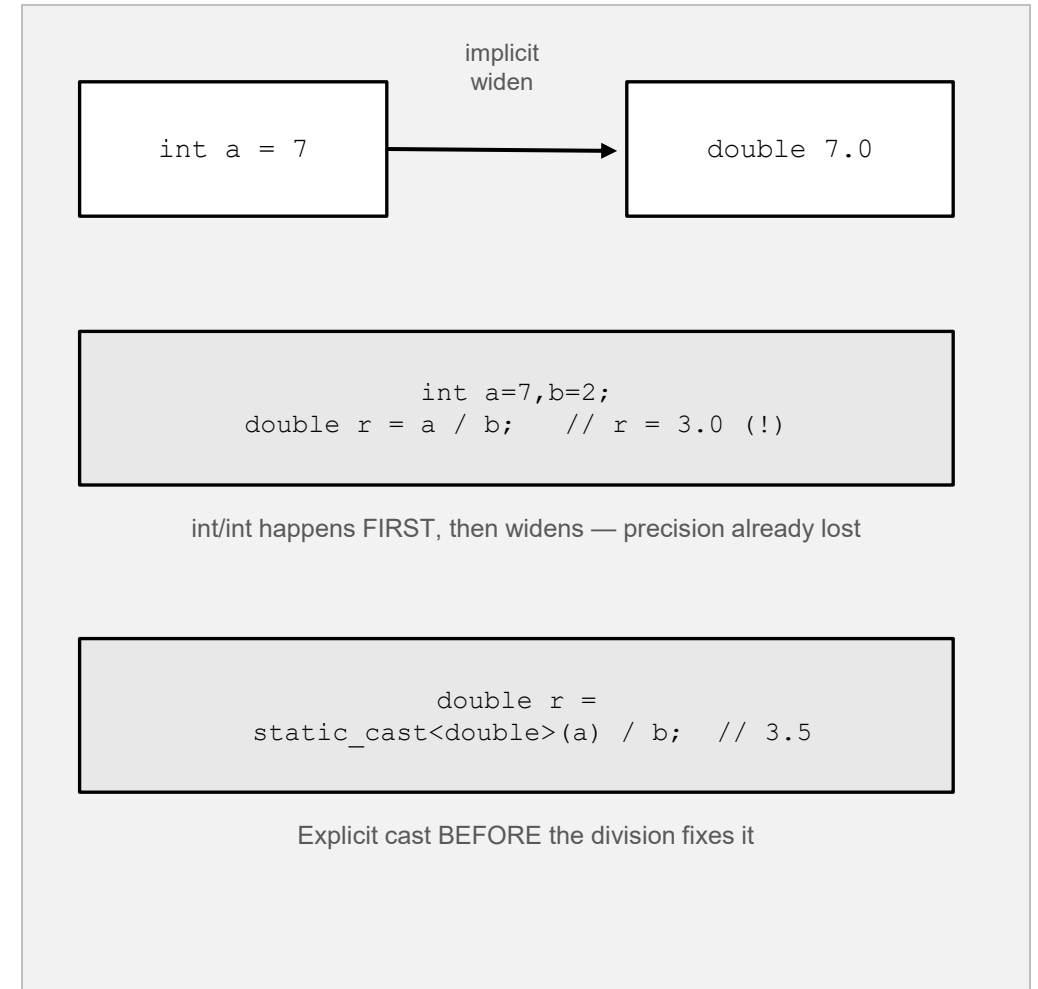
CHAPTER 2

Type Conversion

Definition: Automatic (implicit) conversion happens when the compiler silently converts one type to another because they appear together in an expression — usually widening (int to double), so no information is lost. Explicit conversion (a cast) is when the programmer forces a conversion, most safely with `static_cast<Type>(value)`.

- Implicit: `double d = someInt;` (compiler converts automatically)
- Implicit danger: `int / int` always gives an `int` result, even if assigned to a `double`
- Explicit (C-style): `(double) x` — still works, but not type-checked
- Explicit (C++): `static_cast<double>(x)` — checked, clearer intent
- Other C++ casts: `const_cast`, `dynamic_cast`, `reinterpret_cast` (advanced use)

Analogy: Implicit conversion is like a shop automatically rounding your change to the nearest coin without asking. An explicit cast is you deliberately asking the cashier for a specific denomination.



CHAPTER 2

Type Conversion

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    double b = 3.5;

    double result = a + b;    // automatic (implicit) conversion: a promoted to double
    cout << "Implicit: " << result << endl;    // 13.5

    double pi = 3.14159;
    int truncated = static_cast<int>(pi);    // explicit cast
    cout << "Explicit cast: " << truncated << endl;    // 3

    return 0;
}
```


CHAPTER 2

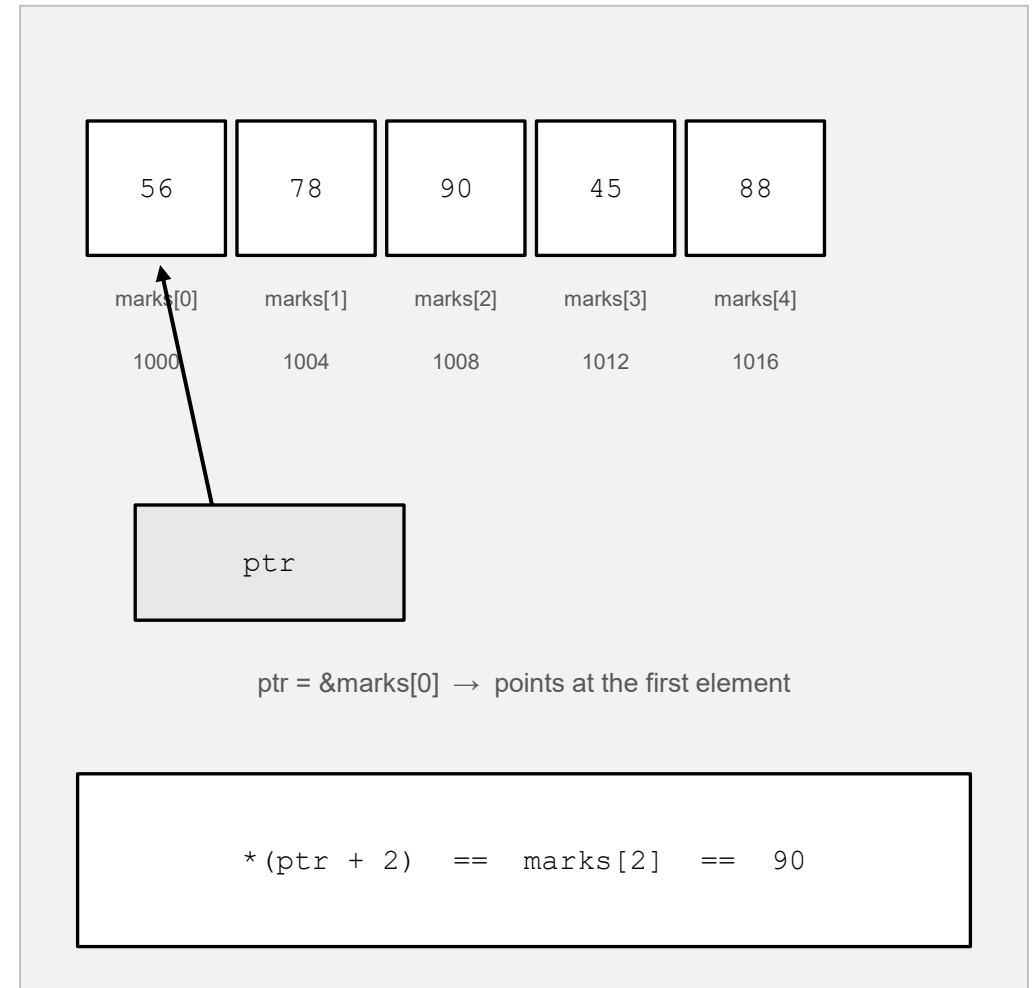
Arrays and Pointers

Definition:

- An array is a fixed-size, contiguous block of elements of the same type, accessed by index.
- A pointer is a variable that stores a memory address.
- An array's name, in most expressions, decays into a pointer to its first element — so `arr[i]` and `*(arr + i)` mean the same thing.

- `int marks[5];` declares 5 contiguous ints
- `int *ptr = marks;` `ptr` now holds the address of `marks[0]`
- `marks[i]` is exactly equivalent to `*(marks + i)`
- Pointer arithmetic moves in units of the pointed-to TYPE's size, not raw bytes
- Pointers can also point to dynamically allocated memory (Topic 5)

Analogy: An array is a street of numbered houses built one after another. A pointer is a slip of paper with just one address on it — you can 'walk' to the next house by adding to that address.



CHAPTER 2

Arrays

- An array is a **fixed-size, contiguous block of memory**.
- All elements in an array must be of the **same data type**.
- Elements are accessed using an **index**, e.g., arr[0], arr[1], and so on.

Definition

An array in C++ is a collection of elements of the same data type, stored in contiguous memory locations. These elements are accessed using an index that starts from 0. C++ arrays provide a structured way to store and manage data, making it easier to work with multiple values of the same type.

Syntax:

```
data_type array_name[array_size];
```

```
#include <iostream>
using namespace std;

int main() {
    int numbers[] = {1, 2, 3, 4, 5};

    int arraySize = sizeof(numbers) / sizeof(numbers[0]);

    cout << "Iterating through the array:" << endl;

    for (int i = 0; i < arraySize; i++) {
        cout << "Element at index " << i << ": " << numbers[i] << endl;
    }

    return 0;
}
```

CHAPTER 2

Multidimensional Arrays

C++ supports multi-dimensional arrays. A 2D array is essentially an array of arrays, and accessing elements involves specifying both row and column indices.

```
#include <iostream>
using namespace std;

int main() {
    // Defining a 2D array
    int multiDimArr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

    // Iterating through the 2D array and printing its elements
    cout << "2D Array elements:" << endl;

    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            cout << multiDimArr[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

CHAPTER 2

Pointers

In C++, pointers are used to store the memory addresses of variables, allowing for direct access and manipulation of memory. There are several types of pointers, each serving specific purposes:

Declaring pointers in C++

```
data_type* pointer_name;  
OR  
data_type *pointer_name;
```

- *When used in a declaration (`int* ptr;`), it informs the compiler that this variable is a pointer.*
- *When used in an expression (`*ptr`), it dereferences the pointer—i.e., accesses the value stored at the memory address the pointer*

Initializing pointers in C++

1. Assigning an existing variable's address

- The most common way to initialize a pointer is by storing the address of an already declared variable.

```
int num = 20;  
int* ptr = &num;  
cout << *ptr << endl;  
*ptr = 30;  
cout << num << endl;
```

2. Using `nullptr` (Modern C++)

- declare a pointer in advance, even if we don't yet have a variable for it to point to

```
int* ptr = nullptr;  
  
if (ptr != nullptr) {  
    *ptr = 10;  
}
```

Pointers

1. Null Pointer

A pointer that doesn't point to any valid memory address. Initialized with `nullptr` in C++11 or `NULL` in older C++ standards.

Example:

```
int* ptr = nullptr;
```

2. Void Pointer (Generic Pointer)

A pointer that can hold the address of any data type. Cannot be dereferenced directly and must be cast to another pointer type before use.

Example:

```
void* ptr;  
int a = 10;  
ptr = &a; // valid assignment
```

3. Dangling Pointer

- A pointer that points to a memory location that has been freed or deleted.
- Accessing a dangling pointer leads to undefined behavior.

Example:

```
int* ptr = new int(5);  
delete ptr;
```

Pointers

4. Wild Pointer

- A pointer that has not been initialized to a valid address.
- Accessing a wild pointer is also undefined and dangerous.

Example:

```
int* ptr; // uninitialized pointer, potentially a wild pointer
```

5. Smart Pointer

Introduced in C++11 to manage dynamic memory automatically, preventing memory leaks.

Types include:

- `std::unique_ptr`: Exclusive ownership; only one `unique_ptr` can point to a resource.
- `std::shared_ptr`: Shared ownership; multiple pointers can own the same resource.
- `std::weak_ptr`: Non-owning reference to a resource managed by a `shared_ptr`.

6. Function Pointer

- A pointer that points to a function instead of a variable.
- Allows passing functions as arguments or storing functions in data structures.

Example:

```
void (*funcPtr)(int) = &myFunction;
```

Pointers

7. Constant Pointer

- A pointer whose address cannot be changed after initialization.

Example:

```
int a = 10;  
int* const ptr = &a; // ptr cannot point to any other address
```

8. Pointer to Constant

- A pointer that points to a constant value, so the value it points to cannot be modified.

Example:

```
const int* ptr = &a;
```

9. Constant Pointer to Constant

A pointer that is both constant itself and points to a constant value.

Example:

```
const int* const ptr = &a;
```

10. Nullptr (C++11 onwards)

A special type of pointer literal that represents a null pointer, replacing NULL.

Safer and less ambiguous than NULL.

Example:

```
int* ptr = nullptr;
```

Pointers

this Pointer

- The **this** pointer is an **implicit pointer** available in all **non-static** member functions of a class.
- It holds the **memory address** of the object that invoked the member function.
- When a member function is called on an object, the compiler **automatically passes** the object's address as a hidden parameter.
- This hidden parameter is named **this**.
- The **this** pointer is **not explicitly declared** by the programmer—it is provided by the compiler.
- It is **not available** in static member functions, since static functions are not tied to any specific object instance.

Common uses of **this** include:

- Resolving naming conflicts (e.g., when parameter names shadow member variables, using `this->member`).
- Returning the current object from a member function (e.g., `return *this;`).
- Passing the current object to another function (e.g., `someFunction(*this);`).
- The **this** pointer is a **constant pointer** (i.e., `ClassName* const this`), meaning it always points to the same object and cannot be reassigned.
- In a `const` member function, this becomes a **pointer to a constant object** (i.e., `const ClassName* const this`), preventing modification of member variables.

CHAPTER 2

Pointers

```
class Student {
    private:
        int age;
        std::string name;
        double gpa;
    public:
        void regularFunction() {
            // this is of type: Student* const
            this->age = 20;           // Can modify private members
            this->name = "John";     // Can modify private members
            this->gpa = 3.5;         // Can modify private members
        }
        void constFunction() const {
            // this is of type: const Student* const
            // this->age = 25;        // ERROR: Cannot modify in const function
            int currentAge = this->age; // Can read private members
            std::string currentName = this->name; // Can read private members
            std::cout << "Age: " << this->age << std::endl; // Can access for reading
        }
};
```

Pointers

```
class Person {
    private:
        string name;
        int age;
    public:
        Person(const std::string& name, int age) {
            this->name = name;
            this->age = age;
        }
        void displayInfo() {
            std::cout << "Name: " << this->name << std::endl;
            std::cout << "Age: " << this->age << std::endl;
        }
        void updateAge(int newAge) {
            this->age = newAge;
        }
};

int main() {

    Person person("Kapil", 19);
    person.displayInfo();
    person.updateAge(20);
    cout << "After updating age:" << endl;
    person.displayInfo();
    return 0;
}
```

CHAPTER 2

Arrays and Pointers

```
int marks[5] = {56, 78, 90, 45, 88};
int *ptr = marks;          // ptr holds address of marks[0]

for (int i = 0; i < 5; i++) {
    cout << "marks[" << i << "] = " << *(ptr + i) << endl;
}
// *(ptr+i) is exactly equivalent to marks[i]
```

new / delete: C++-native heap allocation. Always pair new[] with delete[].

CHAPTER 2

Memory Management in C++

- Memory in C++ is managed through **allocation** (reserving memory) and **deallocation** (releasing memory) for program data and objects.
- **Static memory allocation** occurs at **compile-time** (e.g., global variables, static variables, stack-allocated local variables).
- **Dynamic memory allocation** occurs at **runtime**, providing greater flexibility in memory usage.
- Dynamic memory is allocated on the **heap** (also called the free store).
- Dynamic allocation uses operators like:
 - ✓ **new (C++ style)**
 - ✓ `malloc()` (C style, but still usable in C++)
- Memory allocated dynamically **persists until explicitly deallocated** by the programmer.
- Deallocation is done using:
 - ✓ **delete (for memory allocated with new)**
 - ✓ `free()` (for memory allocated with `malloc()`)
- Dynamic memory requires **explicit management** to prevent:
 - ✓ **Memory leaks** (allocated memory that is never freed)
 - ✓ **Dangling pointers** (pointers pointing to freed memory)
 - ✓ **Double deletes** (freeing the same memory twice)
- Static memory is **automatically managed** by the compiler and does not require manual deallocation.
- Static memory has a **fixed size** determined at compile time, whereas dynamic memory can be sized at runtime based on program needs.
- Dynamic allocation is essential for creating:
 - ✓ Variable-sized data structures
 - ✓ Objects whose lifetime must outlast the scope in which they were created
 - ✓ Dynamic data structures like linked lists, trees, and graphs

New and Delete Operator

new and **delete** are the C++-native way to manage dynamic (heap) memory. **new** allocates memory for an object or array and, unlike **malloc**, also calls the constructor; **delete** frees the memory and calls the destructor. Every **new** must be matched by exactly one **delete** (and **new[]** by **delete[]**) to avoid memory leaks.

New Operator

- The **new** operator in C++ is used for **dynamic memory allocation**.
- It allocates memory for a **variable** or an **array of variables** at **runtime**.
- Dynamic allocation with **new** provides **flexibility** in managing memory during program execution.
- Memory allocated with **new** **persists** until it is explicitly deallocated by the programmer.
- To free memory allocated with **new**, the **delete operator** must be used.
- If memory allocated with **new** is not deallocated, it leads to a memory leak.
- Using **new** is essential when the size of data is not known at compile-time.
- **new** returns a pointer to the starting address of the allocated memory block.
- For arrays, **new[]** is used, and **delete[]** must be used to deallocate them properly.

```
<data_type pointer_name> = new <data_type>
```

CHAPTER 2

New and Delete Operator

```
#include <iostream>
using namespace std;

int main() {

    cout << "\nPointer with new keyword" << endl;
    cout << "-----" << endl;

    int *ptr = new int; // dynamically allocating memory for an integer
    *ptr = 42; // assigning a value to the allocated memory

    cout << "Value of *ptr: " << *ptr << endl;

    delete ptr; // deallocating the memory

    return 0;

}
```

New and Delete Operator

delete Operator

- The programmer is responsible for deallocating dynamically allocated memory when it is no longer needed.
- When a dynamically allocated variable is no longer required, the programmer can free its memory using the ***delete*** operator.
- The ***delete*** operator releases the memory back to the free store (heap) for reuse.
- This process is known as **memory deallocation**.
- Deallocating memory returns it to the operating system for future use.
- Even if the programmer forgets to deallocate, the memory is **automatically deallocated** when the program terminates.
- Proper use of ***delete*** helps prevent memory leaks and ensures efficient memory management.
- For dynamically allocated arrays, the ***delete[]*** operator must be used to deallocate memory correctly.

```
delete pointer_variable;
```

CHAPTER 2

New and Delete Operator

```
#include <iostream>
using namespace std;

int main() {

    int* arr = new int[5]; // Allocates memory for an array of 5 integers

    for(int i = 0; i < 5; ++i) {
        arr[i] = i + 1; // Assigns values to the array
    }
    for(int i = 0; i < 5; ++i) {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;

    delete[] arr; // Deallocates the memory for the array

    return 0;

}
```


CHAPTER 2

Manipulator

- Manipulators are **functions** used to control the formatting of input/output streams in C++.
- They are declared in the `<iomanip>` header (some basic ones are also available in `<iostream>`).
- Manipulators are applied using the **insertion operator** (`<<`) alongside the stream.
- They **change the formatting state** of the stream, affecting how subsequent data is displayed.

Common manipulators include:

- ✓ `endl` – Inserts a newline character and **flushes** the output buffer.
- ✓ `setw(n)` – Sets the **field width** to `n` characters for the **next** item only.
- ✓ `setprecision(n)` – Sets the **number of digits** shown for floating-point numbers.
- ✓ `fixed` – Forces floating-point numbers to be displayed in **fixed-point notation** (e.g., 123.45 instead of 1.2345e+02).
- ✓ `setfill(c)` – Sets the **padding character** to `c` when the output field width is larger than the content.
- ✓ Some manipulators (like `fixed` and `endl`) are **sticky** – they remain in effect until changed.
- ✓ Others (like `setw`) are **non-sticky** – they apply only to the very next output operation.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double pi = 3.14159265;
    cout << fixed << setprecision(2);
    cout << setw(10) << "Pi is" << setw(8) << pi << endl;
    return 0;
}
```

CHAPTER 2

Constant

The `const` keyword in C++ is a **type qualifier** that makes a variable or object **immutable** (read-only). It's a promise to the compiler that the value won't change after initialization.

Why Use `const`?

Benefit	Explanation
Safety	Prevents accidental modification
Clarity	Documents intent in code
Optimization	Enables compiler optimizations
Correctness	Enforces design constraints
Thread Safety	Immutable objects are thread-safe

Basic Syntax

```
const int MAX_SIZE = 100;
const float PI = 3.14159;
const string NAME = "Saroj Dahal";
```

Must be initialized

```
const int x;           // ❌ ERROR: uninitialized const
const int y = 10;      // ✅ Correct: initialized
```

CHAPTER 2

Constant

```
#include <iostream>
using namespace std;

class student {
public:
    int n;
    // Constructor
    student(int a) : n(a) {}

    // Const member function to display n
    void show() const {
        cout << n << endl;
    }

    // Modify function (corrected logic)
    void modify(int a) {
        n = a; // Assuming 'q' was meant to be 'n'
    }
};

int main() {
    // Create student object with value 50
    student s1(10);
    s1.modify(20); // Modify n to 20

    student s2(30); // Const object can only call const functions
    s2.show();

    return 0;
}
```

CHAPTER 2

Enumeration

- An **enumeration** (enum) is a user-defined data type that consists of a set of **named integral constants**.
- It allows you to assign meaningful names to integer values, making code more **readable** and **maintainable**.
- Enums are defined using the enum keyword.

Syntax

```
enum EnumName {  
    CONSTANT1,  
    CONSTANT2,  
    CONSTANT3  
};
```

Important Characteristics

- By default, the first enumerator has the value **0**, and each subsequent one increases by **1**.
- You can explicitly assign values to enumerators.
- Enumerators are **compile-time constants**.
- Enumeration types are **distinct types** in C++ (stronger type checking than in C).
- Enums can be used in **switch statements**, **loops**, and as **function parameters**.

CHAPTER 2

Enumeration

```
#include <iostream>
using namespace std;

// Define an enumeration for days of the week
enum Days {
    Monday,    // 0
    Tuesday,   // 1
    Wednesday, // 2
    Thursday,  // 3
    Friday,    // 4
    Saturday,  // 5
    Sunday     // 6
};

int main() {
    // Declare a variable of type Days
    Days today = Wednesday;

    cout << "Today is: " << today << endl;    // Outputs integer value (2)
    cout << "Wednesday has value: " << Wednesday << endl;

    // Using enum in a loop
    for (int i = Monday; i <= Sunday; i++) {
        cout << "Day " << i << endl;
    }

    return 0;
}
```

Practice & Exam-style Questions

1. Explain implicit vs explicit type conversion with an example.
2. Explain new and delete with a program; compare to malloc/free.
3. What is the relationship between arrays and pointers in C++? Demonstrate with a program that reverses an array using pointer arithmetic only (no indexing).
4. Differentiate #define and const.
5. Explain any three manipulators with examples.



Probable Exam Question

Q1: What is type casting? Explain with an example the difference between implicit and explicit type conversion.

Q2: Explain reference variables and scope resolution operator with examples. Write a program using setw, setprecision, and setfill. [PU Exam 2021]

CHAPTER 3

Functions Used in C++

Passing arguments · Return by reference · Overloading · Default arguments · Inline

Functions

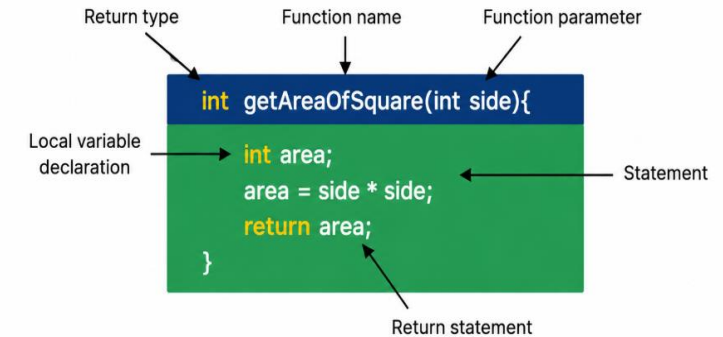
A **function** is a group of statements that together perform a task. Every C++ program has at least one function, `main()`, which is the starting point of the program. Functions can be defined by the user or provided by libraries.

Why Use Functions?

- **Modularity:** Break down complex problems into smaller, manageable pieces.
- **Reusability:** Write code once and use it multiple times, even in different programs.
- **Organization:** Make the code more organized and easier to read and maintain.

Syntax:

```
returnType functionName(parameterList) {  
    // body of the function  
}
```



Here,

- **returnType:** The data type of the value the function returns. If the function does not return a value, this is specified as `void`.
- **functionName:** The name of the function.
- **parameterList:** A list of parameters (arguments) that the function takes, enclosed in parentheses. This is optional; a function may have no parameters.

Functions

Methods To Call A C++ Function

There are two primary ways to call the C++ function inside a program with access.

- call-by-value
- call-by-reference

Call-By-Value

The call-by-value technique allows the arguments to be sent to a C++ function by making a duplicate of the values and giving them to the function.

```
void increment(int num) {  
    num++; // Increment the local copy of num  
}  
  
int main() {  
  
    int value = 5;  
    std::cout << "Before function call: " << value << std::endl;  
  
    increment(value); // Call the function by value  
    std::cout << "After function call: " << value << std::endl;  
  
    return 0;  
}
```

CHAPTER 3

Functions

Call-By-Reference

The call-by-reference way of passing arguments to a C++ function is when the function is called by supplying the memory location (reference) of the variables to be utilized inside the function.

```
#include <iostream>
using namespace std;

void increment(int& num) {
    num++; // Increment the original value referred to by num
}

int main() {

    int value = 5;
    std::cout << "Before function call: " << value << std::endl;

    increment(value); // Call the function by reference

    std::cout << "After function call: " << value << std::endl;

    return 0;
}
```

Functions

Types of Function:

1. Standard Functions

Standard functions are those provided by C++ libraries, such as `std::cout` for output and `std::cin` for input

2. User-Defined Functions

User-defined functions are written by the programmer to perform specific tasks. The example above illustrates a simple user-defined function.

Function Components

- **Return Type**

A function may return a value. The return type is the type of value the function sends back to the caller. If a function does not return a value, its return type is void.

- **Parameters**

Parameters (or arguments) are the values passed to the function. They are optional; a function can have none.

Parameters allow you to pass data into a function.

- Pass-by-value
- Pass-by-reference

- **Function Body**

The function body contains the statements that define what the function does. It's enclosed in curly braces {}.

Functions

Passing Parameters to a function

Pass-by-Value Approach For C++ Function

When we are passing parameters to a function using the pass-by-value approach, a copy of the argument is provided to the function. The clone of the argument is used by the function, and any changes performed there have no impact on the original argument.

```
#include <iostream>
using namespace std;

void increment(int num) {
    num++;
}

int main() {

    int value = 7;
    std::cout << "Before function call: " << value << std::endl;

    increment(value);
    std::cout << "After function call: " << value << std::endl;

    return 0;
```

Functions

Passing Parameters to a function

Pass-by-Reference Approach For C++ Function

Under the pass-by-reference, a reference to the argument is supplied to the C++ function. The original parameter can be directly accessed and changed by the C++ function. The ampersand (&) is used in function declarations to denote a reference before an argument

```
#include <iostream>
using namespace std;

void increment(int& num) {
    num++;
}

int main() {

    int value = 8;
    std::cout << "Before function call: " << value << std::endl;

    increment(value);
    std::cout << "After function call: " << value << std::endl;

    return 0;
}
```

Functions

Swap by Reference

```
#include <iostream>
using namespace std;

void swapNumbers(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10;
    int y = 20;

    cout << "Before swapping:" << endl;
    cout << "x = " << x << ", y = " << y << endl;

    swapNumbers(x, y);

    cout << "After swapping:" << endl;
    cout << "x = " << x << ", y = " << y << endl;

    return 0;
}
```

Functions

Function Parameters: The Basics

When a function is called, often data are provided for function to work with. These data are passed as **parameters** to the function.

- **Arguments:** The actual values passed in the function call.
- **Parameters:** The variables defined in the function that receive the arguments.
- Parameters act as **placeholders** that hold the values passed from the calling code.
- The data flows **from arguments (caller) to parameters (function)** when the function is invoked.
- Parameters are **local to the function** and exist only within its scope.
- The number and type of arguments must **match** the number and type of parameters declared in the function.
- Arguments can be passed as **values, references, or pointers** depending on the function definition.

Functions

Default Argument:

- **Default arguments** allow us to specify **default values** for function parameters in C++.
- These default values are used **automatically** when the corresponding argument is **omitted** in the function call.
- Default arguments enhance **function flexibility** by accommodating multiple usage scenarios.
- They **simplify function calls** by reducing the number of arguments that must be explicitly passed.
- Default arguments maintain **code clarity** by minimizing the need for multiple overloaded functions.
- They are defined in the **function declaration** (prototype), not in the definition.
- Default arguments must be specified from the **rightmost parameter** (trailing parameters) towards the left.
- Once a parameter has a default value, **all subsequent parameters** must also have default values.
- They are useful when a function has **commonly used values** but still allows customization.
- Default arguments reduce **code duplication** by eliminating the need for separate functions for different use cases.

Syntax:

```
returnType functionName(parameterType1 parameterName1 = defaultValue1, ...);
```


CHAPTER 3

Functions

```
#include <iostream>
using namespace std;

// defining the default arguments
void display(char c = '*', int count = 3) {
    for(int i = 1; i <= count; ++i) {
        cout << c;
    }
    cout << endl;
}

int main() {

    int count = 5;

    cout << "No argument passed: ";
    display();

    cout << "First argument passed: ";
    display('#');

    cout << "Both argument passed: ";
    display('$', count);

    return 0;
}
```

```

class Rectangle {
private:
    int width;
    int height;

public:
    Rectangle(int w = 0, int h = 0) : width(w), height(h) {
        cout << "Constructor called" << endl;
    }

    Rectangle(const Rectangle& other) {
        width = other.width;
        height = other.height;
        cout << "Copy constructor called!" << endl;
    }

    int area() const {
        return width * height;
    }

    void setWidth(int w) {
        width = w;
    }
};

// Function that takes Rectangle by value (creates a copy)
void modifyRectangleByValue(Rectangle r) {
    r.setWidth(20); // Modifies the copy, not the original
    cout << "Inside function area: " << r.area() << endl;
}

int main() {

    Rectangle rect(5, 10);
    cout << "Original area: " << rect.area() << endl;

    modifyRectangleByValue(rect); // Copy is created

    cout << "Original area after function: " << rect.area() << endl;

    return 0;
}

```

Functions Overloading

Multiple Functions with same name but different parameters

Function overloading is a feature in C++ that allows multiple functions to have the same name but with different parameters (either in type, number, or both). This capability enhances the expressiveness and readability of the programming language, allowing functions to be used in various contexts with different types of input.

```
#include <iostream>
using namespace std;

// Overloaded functions to calculate the area
double area(double radius) { // For circle
return 3.14159 * radius * radius;
}

double area(double length, double width) { // For rectangle
return length * width;
}

int main() {
cout << "Area of circle (r=5): " << area(5.0) << endl;
cout << "Area of rectangle (4x5): " << area(4.0, 5.0) << endl;
return 0;
}
```

Explanation of Program:

```
double area(double radius) {
```

This version of the area function calculates the area of a circle. It takes one double parameter representing the radius of the circle and returns its area as a double.

```
double area(double length, double width) {
```

This version calculates the area of a rectangle. It is overloaded to take two double parameters representing the length and width of the rectangle, respectively, and returns the calculated area.

```
cout << "Area of circle (r=5): " << area(5.0)
<< endl;
```

Calls the first version of the area function with a single argument to calculate the area of a circle.

```
cout << "Area of rectangle (4x5): " <<
area(4.0, 5.0) << endl;
```

Calls the second version of the area function with two arguments to calculate the area of a rectangle.

Functions Overloading

Use Cases

Handling Different Data Types: Overloaded functions can perform analogous operations on different types of data. For example, you might have a print function that accepts both int and float types, displaying numbers in an appropriate format.

Different Implementations Based on Parameters: You can overload functions to provide different implementations based on the number or types of arguments passed. This is useful in mathematical operations, where the formula might differ based on input dimensions or types.

Improving Code Readability: Using the same function name for similar operations on different data types makes the code more intuitive and easier to understand.

Advantages

Cleaner Code: Function overloading leads to cleaner code by reducing the number of distinct function names for similar operations, making the code easier to read and maintain.

Intuitive Usage: Overloading allows functions to be called in a way that feels natural and intuitive, closely mimicking the versatility seen in human languages.

Compile-Time Polymorphism: It provides a form of polymorphism where the appropriate function is selected at compile time based on the argument types, improving performance by avoiding runtime decision-making.

Inline Function

Inline functions are functions in C++ that are expanded in-line at the point of each call, replacing the function call with the actual function code, thereby eliminating the overhead of a traditional function call.

- Inline functions are a **performance optimization feature** designed to improve execution speed.
- They are most effective for **small, frequently called functions**.
- By marking a function with the inline keyword, the programmer gives a **hint** to the compiler to embed the function's body directly at each call site.
- Instead of a **traditional function call** (which involves stack operations, jump, and return), the compiler **replaces the call with the function code** itself.
- This eliminates the **overhead** of function call mechanisms (saving time and resources).
- Inline functions **originated** from C's macro feature but provide **type safety** and better debugging support.
- They are particularly useful for **getter/setter functions, mathematical operations, and small utility functions**.
- The inline keyword is only a **request**, not a command – the compiler may ignore it if the function is too large or complex.

Potential risks include:

Code bloat – excessive inlining can increase the size of the executable significantly.

Cache misses – larger code size may reduce CPU cache efficiency.

Increased compilation time – due to repeated expansion of code.

Debugging difficulty – as the function call stack may not reflect actual calls.

Overuse can lead to **diminished performance** rather than improvement.

CHAPTER 3

Inline Function

```
#include <iostream>
using namespace std;

inline int square(int x) {
    return x * x;
}

int main() {
    int result = square(5); // Compiler may replace this with: int result = 5 * 5;
    return 0;
}
```

Practice & Exam-style Questions

1. What is function overloading? Overload `add()` for `int` and `float`.
2. Explain call-by-value vs call-by-reference with an example.
3. What are default arguments? Explain the rules.
4. Define inline function. Why and when is it used?



Probable Exam Question

Q1: What are inline functions? Write a C++ program demonstrating function overloading and default arguments. [PU Exam 2023]

Lab Work

Scenario 1: Smart Calculator

Problem Statement

Develop a calculator program that can perform addition on different types of inputs.

Requirements

Create overloaded functions named add() for:

- *Two integers*
- *Two floating-point numbers*
- *Three integers*

Create an inline function square() that returns the square of a number.

Display the result of each operation.

Concepts needed to implement

- *Function Overloading*
- *Inline Function*
- *Basic Arithmetic Operations*

Lab Work

Scenario 2: Employee Salary Management

Problem Statement

A company wants a salary calculation system for different categories of employees.

Requirements

Overload a function calculateSalary() for:

- *Full-time employee (basic salary)*
- *Employee with bonus*
- *Employee with bonus and overtime*

Create an inline function taxDeduction() that calculates 10% tax.

Display gross salary and net salary.

Concepts needed to implement

- *Function Overloading with different parameter lists*
- *Inline Functions*
- *Real-world business application*

Lab Work

Scenario 3: Student Result Processing

Problem Statement

Create a result-processing system for students.

Requirements

Overload a function calculateAverage() for:

Two subjects

Three subjects

Five subjects

Create an inline function isPass() that returns true if average ≥ 40 .

Display average marks and pass/fail status.

Concepts needed to implement

- *Function Overloading with different parameter lists*
- *Inline Functions*
- *Real-world business application*

CHAPTER 4

Classes and Objects

Access specifiers · this pointer · Constructors & destructors · Static members · Friend functions

The Architecture of C++ Classes and Objects

A visual schematic of Object-Oriented Programming mechanics, memory structures, and access controls.

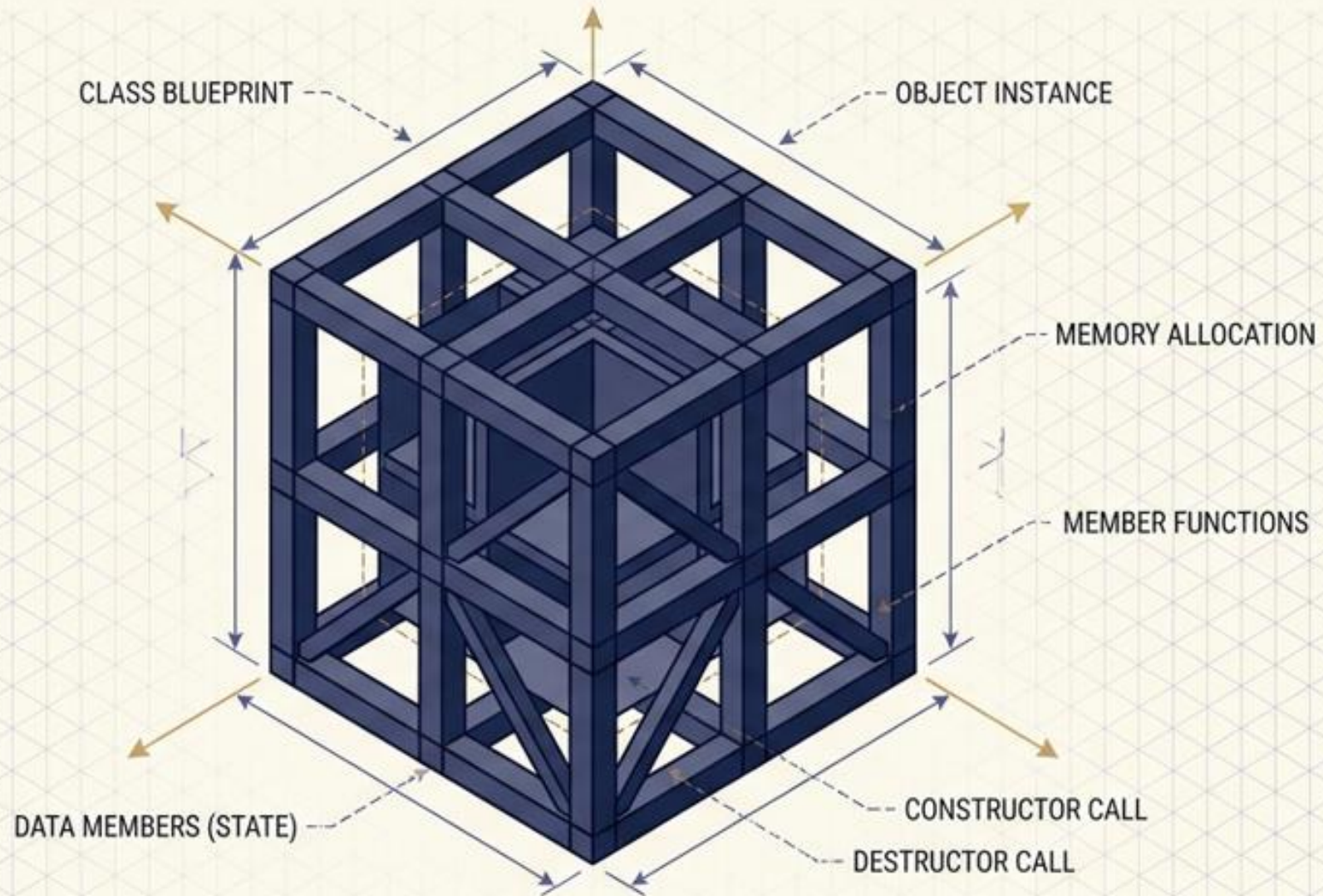


Figure 1.1: Isometric Schematic of a C++ Class and its Object Instantiation Process.

Introduction

- Objects and Classes are key to understand the Object-Oriented Programming Language (OOPL)/Object-Oriented Technique. Object-Oriented Programming Languages are based on the concept of abstraction modeled by classes and objects.
- Classes and objects are the cornerstones of Object-Oriented Programming (OOP) in C++. They enable programmers to create modular, reusable, and maintainable code. In this article, we will dive deep into understanding what classes and objects are, explore their functionalities, and learn how to implement them in C++ effectively.
- Classification is one of the important concepts of Object-Oriented philosophy including identity, abstraction, encapsulation, inheritance, polymorphism, and persistence.

Object-Oriented uses classification to group objects that have attributes and behaviors in common and classify them into bigger entities. The bigger entity is called a “class”. Thus, a class defines a group of objects with similar attributes, common operations, a common relationship to the other objects in the class, and common semantics

Class

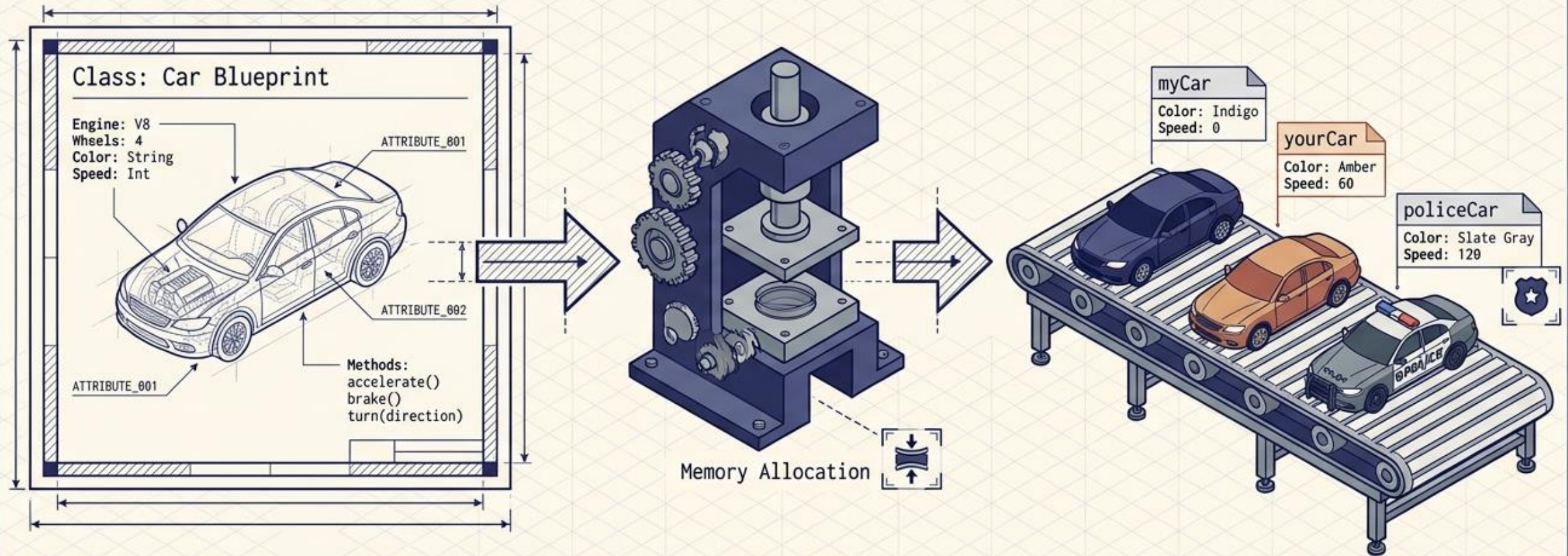
A class is set of objects that shares a common definition described by data and methods.

According to the Webster's New World dictionary, a class is defined as:

“A number of people or things grouped together because of certain likeness; kind; sort”. In other words, we can say that class is an object template. Every object under that class has the same data format, definition and responds in the same manner to an operation.

A **class** in C++ is a user-defined data type that encapsulates data members (variables) and member functions (methods) within a single unit. Think of a class as a blueprint or template that defines the structure and behavior of objects. Classes are the foundation of OOP and facilitate encapsulation, abstraction, and data hiding.

The Blueprint and the Manufactured Product



Class (The Blueprint)

A user-defined data type. It is a logical template that encapsulates data (**attributes**) and member functions (**behaviors**) into a single unit. It consumes no memory on its own.

Object (The Product)

An active instance of a class. When an object is **instantiated**, **memory is allocated**, and it becomes a physical entity in the program that holds unique data.

Class

Analogy of Classes:

Imagine a class of cars. Cars can have different names, models, and brands, yet they all share common properties like having four wheels, a speed limit, and mileage. In this analogy:

- ✓ The **Car** represents the class.
- ✓ The common properties such as wheels, speed limits, and mileage are the **data members**.
- ✓ Actions like accelerating, braking, and honking are the **member functions**.

Defining a Class in C++

A class in C++ is defined using the class keyword followed by the class name. The body of the class contains the data members and member functions, enclosed in curly braces {}.

Example

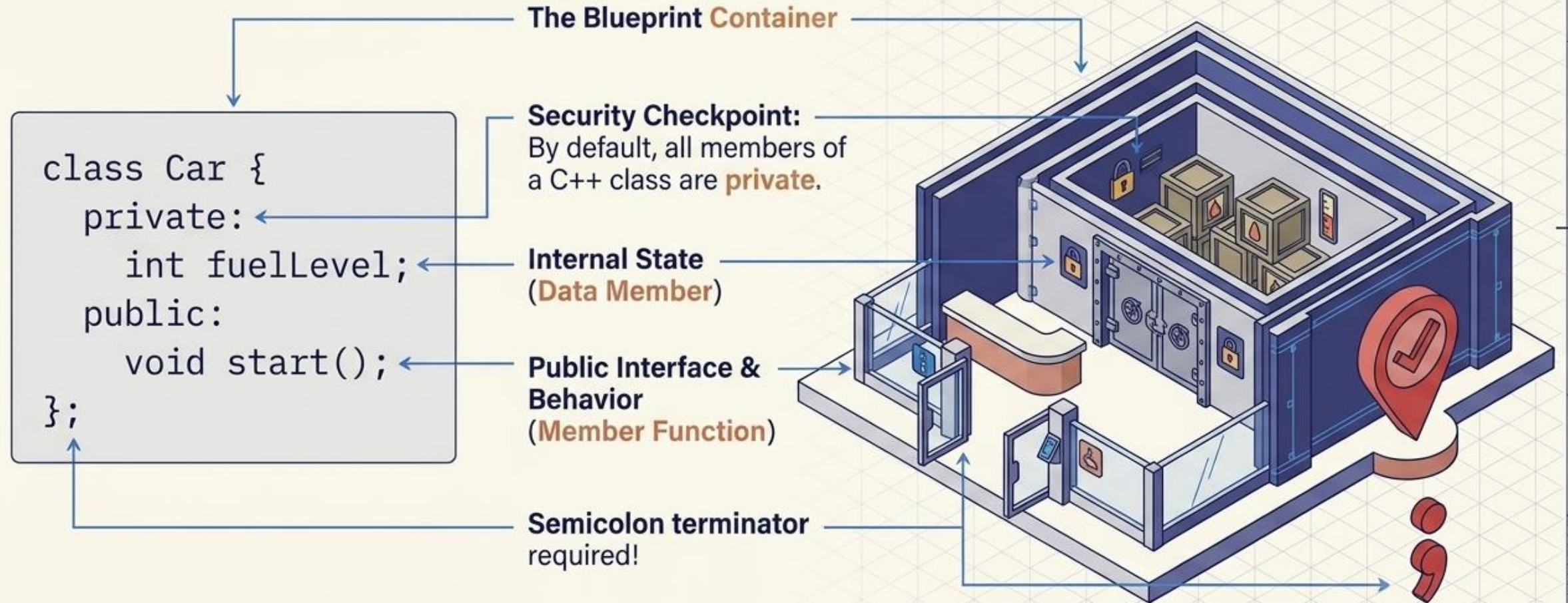
Class: Expensive Cars

Objects: Mercedes, BMW, Toyota

Data Member: Price, Speed, Wheels, Mileage, Model, Color

Member Function: brakes(), speed(), start(), stop(), etc.

Anatomy of a C++ Class



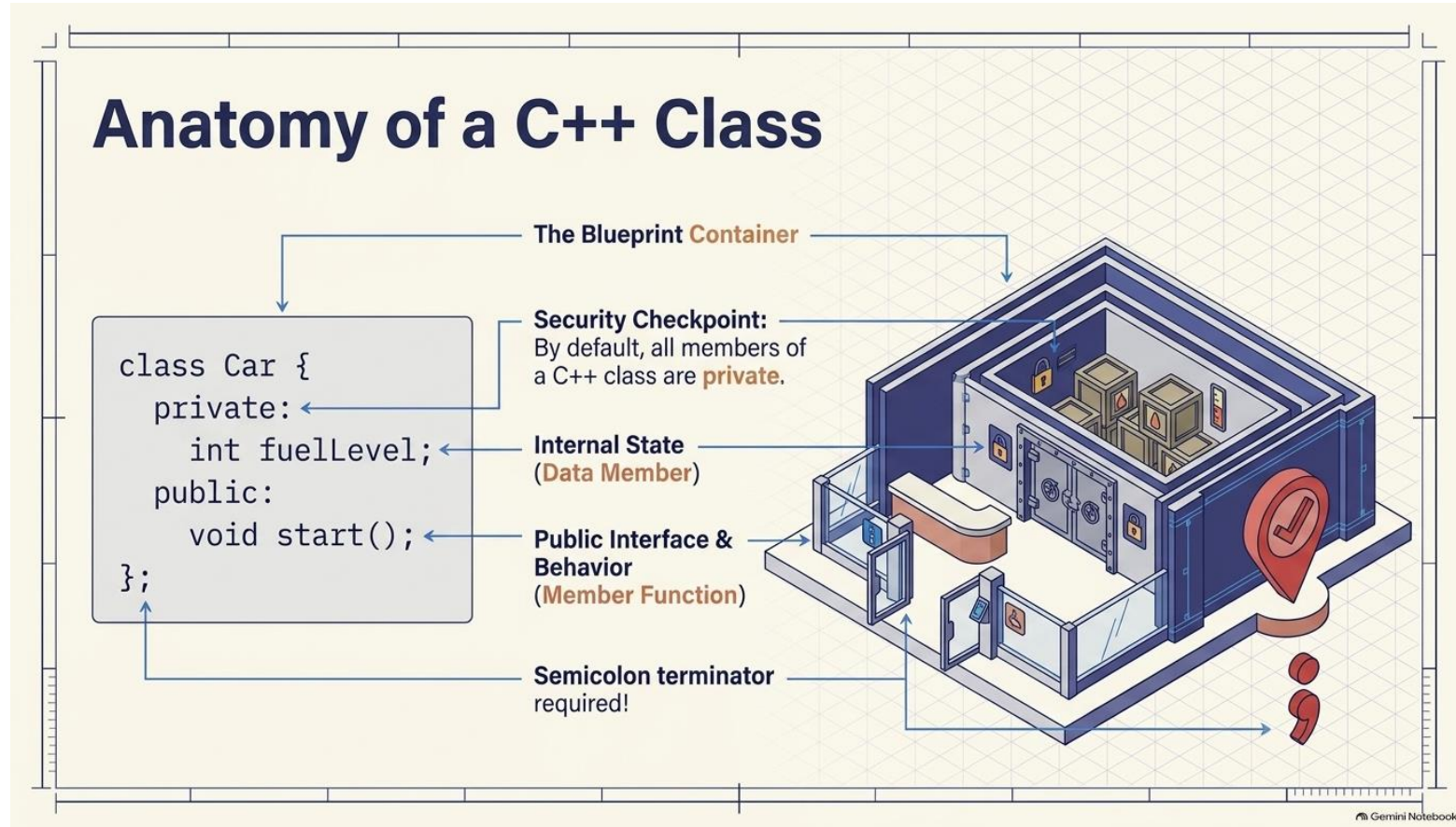
CHAPTER 4

Class

Syntax:

```
class ClassName {  
    access_specifier:  
    // Data_members;  
    // Member_Functions()  
};
```

- The **keyword class** is used to define a class in C++, where the class identifier is given by **ClassName**.
- The **access_specifier** is the access controller which defines the visibility of class members. It can be private, public, and protected; we will elaborate on these in a later section.
- **Data_members** are the variables that hold data related to an object of the class.
- **Member_functions()** are class methods that define what operations can be performed on objects and how.



CHAPTER 4

Objects

An **object** is anything in the real world that has a distinct identity, observable characteristics, and defined behaviors.

Think of it like a **smartphone**:

State (Attributes): Brand, model, color, battery level.

Behavior (Methods): Making a call, sending a text, or opening an app.

According to the Webster's New World dictionary, an Object is defined as:

"A thing that can be seen or touched; material thing; a person or thing to which action, thought or feeling is directed".

An **object** is an instance of a class. When an object is created, a real-world entity is created that has the properties and behaviors defined by the class. While a class only defines what an object will be, an object is an actual instance that occupies memory and can be manipulated.

Creating an Object:

To use the data members and functions of a class, you need to create an object. When you create an object, memory is allocated for it, allowing you to access and manipulate the data members and functions defined in the class.

Syntax:

```
ClassName ObjectName; //Creating an object
```

- **Class_name:** It is the name of the class for which the object is created
- **Object_name:** It is the name of the object. Note that having a meaningful object name enhances code readability

CHAPTER 4

Objects

1. **Declare and Define a Class in C++ Program:** The first step is to create a class, as mentioned in the syntax given above. The C++ class definition must specify-
 - ✓ Data members, i.e., the attributes or member variables of objects in C++.
 - ✓ Member functions, i.e., C++ class methods that define how we can manipulate the data stored in the member variables to full required tasks.
2. **Create an Object in C++ Program:** After a class has been defined, an object of that class can be declared by writing the class name and then the object name. The memory for the object is assigned after this declaration.
3. The creation of a C++ class object is usually done in the main() part of the program.
4. Once declared and initialized, the C++ class object can be used to invoke special functions in the class.

```
#include <iostream>
using namespace std;
// Define a class called 'Car'

class Car {
    public:        // Access specifier
        string brand; // Data member
        int speed;    // Data member

        void display() {        // Member function
            cout << "Brand: " << brand << ", Speed: " << speed << " km/h" << endl;
        }
};

int main() {
    // Creating an object of the Car class
    Car myCar;

    // Setting values for the object's data members
    myCar.brand = "Toyota"; // Assign brand name
    myCar.speed = 120;      // Assign speed value

    // Displaying the car details
    myCar.display(); // Call the display function to output car information

    return 0; // Indicate successful program completion
}
```


CHAPTER 4

Data Member

A **data member** is a variable declared within a class or struct that holds data (state) for objects of that class type. It represents the attributes or properties of the class.

Formal Definition

A **data member** (also called a field, attribute, or instance variable) is a non-function member of a class that stores data. It can be of any type, including fundamental types, pointers, references, arrays, or other class types. Each data member has storage associated with it for each object instance (except static members, which are shared).

```
#include <iostream>
class ClassName {
    // Data member declaration syntax:
    [access_specifier:] [modifiers] type name [= initializer];

private:
    int age;                // simple data member
    std::string name;       // object data member
    static int count;       // static data member
    const double PI = 3.14159; // const data member
    int* ptr;               // pointer data member
    int& ref;               // reference data member (must be initialized)
    mutable int cache;      // mutable data member
    int arr[10];            // array data member
    std::vector<int> scores; // container data member
};
```

Data Member

Type of Data Member

Type	Declaration	Storage	Example
Instance	type name;	Per object	int x;
Static	static type name;	Shared across all objects	static int count;
Const	const type name;	Per object, cannot modify	const int id;
Reference	type& name;	Per object, must bind	int& ref;
Mutable	mutable type name;	Can modify in const methods	mutable int cache;
Bit-field	type name : bits;	Space-efficient	int flag : 1;

Member Function

A function that is defined as a class member is known as a **member function** in C++ programming.

Characteristics of Member Function

- **Member functions** are functions defined within a class.
- They can access the values of the data members and can also carry out operations on the data members.
- Their definition may be included in or excluded from the class body. A member function in C++ can access both public and private class members.
- Member function makes the program modular by increasing code reusability and making it easier to understand, hence making the code maintainable.
- Member functions also provide the ability to restrict access to the data of members of the class. This can be done by making specific functions private, thus ensuring that only the class itself may access or modify sensitive data.
- Member functions are essential in C++ as they encapsulate the behavior and functionality of objects. They enable objects to interact with their data and serve as a means for other components of the program to modify the data.

```
// Member function declaration
returnType functionName (arguments)
{
    //function body
};
```

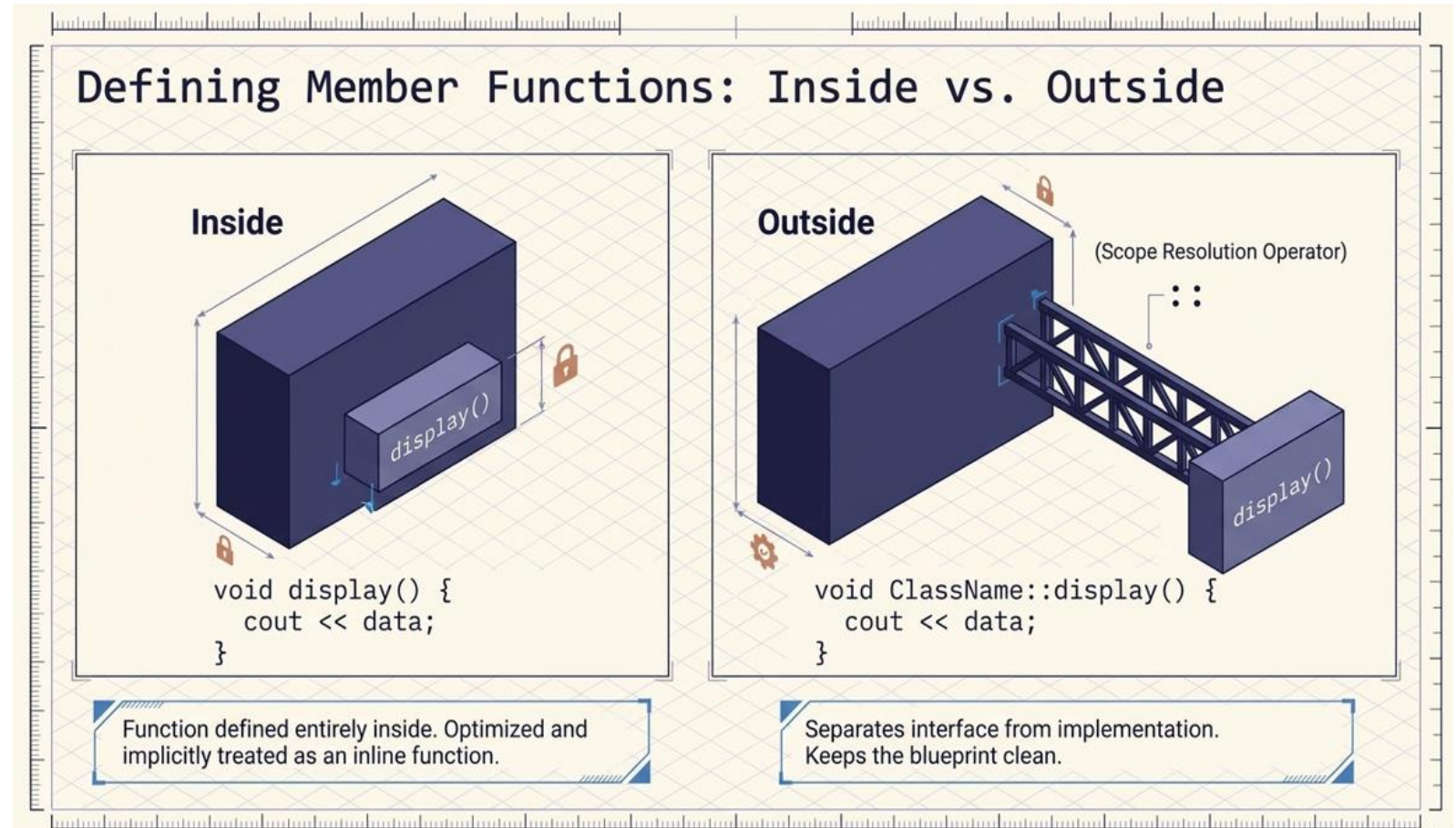
- The **functionName** refers to the name of the function, with the **returnType** specifying the data type of its return value.
- The **arguments** refer to the parameters the member function can take.
- The **curly brackets {}** contain the function body, i.e., the operations that are performed on the data members of the class.

Member Function

Defining a Function in C++

There are mainly two ways to define a member function in C++.

- Inside the class definition
- Outside the class definition.



CHAPTER 4

Member Function

Defining a Function in C++ Inside The Class Definition

A member function can be both declared and defined within a class itself. When a member function is defined within a class, it is referred to as an inline function. All of the constraints and limitations that apply to inline functions also apply to functions declared within a class.

```
class MyClass{
    private:
        //Data Members variables

    public:
        //Member Function
        returnType functionName(argument){
            //Function Definition
        }
};
```

CHAPTER 4

Member Function

```
#include <iostream>
using namespace std;

class Circle{
    private:
        //Data Members
        float rad;

    public:
        //Member Function
        void getradius(){
            cout<<"Enter Radius"<<endl;
            cin>>rad;}

        void calc_area(){
            float a;
            a = 3.14*rad*rad;
            cout<<"The Area of the Circle is = "<<a<<endl;
        }
};

int main(){
    Circle c;
    c.getradius();
    c.calc_area();
    return 0;
}
```

- **Circle class** is defined , which has a **private data member** called **rad** (of float type).
- In the class is the member function called **getradius()**, which prompts the user to enter the radius of the circle **using cout**, stores it in the rad data member, and reads the input using the **cin command**.
- Also, another void member function called **calc_area** within the Circle class. It calculates the area of the circle using the formula $a = 3.14 * rad * rad$ and displays it on the console.
- In the **main() function**, we first create an **object c** of the Circle class.
- We then **call the getradius()** member function on the c object, which prompts the user for input and stores the radius in the rad data member. The **calc_area()** member function then calculates the area of the circle and displays it on the console.

Member Function

Defining a Function in C++ Outside The Class Definition

In this case, the declaration and definition of the member function are not made together. That is, while the functions are declared inside the class, their definition is given outside.

However, the definitions of member functions are very similar to those of normal functions. They should have a function header and body. Unlike a normal function, a member function includes a membership identity label in the header. The identity label is used to indicate that the function belongs to a particular class.

```
return_type class_name :: function-name (argument declaration){  
    //Function body  
}
```

- The **class_name** is the membership label or the name of the class to which the member function belongs.
- The **function_name** and the double colon (::) are the name of the function and the scope resolution operator, which tells the compiler that the function belongs to the class_name.
- The **arguments** and the **return_type** refer to the parameters the function takes and the data type of its return value, respectively.

CHAPTER 4

Member Function

```
#include <iostream>
using namespace std;

class Circle{
private:
float rad; //Data Members

public:
void getradius(); //Member Function
void calc_area(); //Member Function
};

void Circle :: getradius(){
cout<<"Enter Radius ";
cin>>rad;}

void Circle :: calc_area(){
float a;
a = 3.14*rad*rad;
cout<<"The Area of the Circle is = "<<a<<endl;}

int main(){
    Circle c;
    c.getradius();
    c.calc_area();
    return 0;
}
```

- **Circle class** is defined , which has a **private data member** called **rad** (of float type).
- It contains the member functions called **getradius()**, **calc_area()** which are defined outside the class using scope resolution operators (::)
- In the **main() function**, we first create an **object c** of the Circle class.
- We then **call the getradius()** member function on the c object, which prompts the user for input and stores the radius in the rad data member. The **calc_area()** member function then calculates the area of the circle and displays it on the console.

Static Data Member

Introduction

- Declared using the keyword `static` inside the class.
- Must be defined outside the class .
- Shared by all the objects of the class.
- Accessed using the class name followed by the scope resolution operator(`::`).
- Can initialize a static data member in C++ programs using a constant expression or a compile-time constant

A static data member in C++ refers to a member of a class that is common to all its objects. In other words, it is associated with the class as a whole. These members are created using the `static` keyword.

Static Data Member In C++?

The static data member in C++ programming is a special type of class member that is associated with the class rather than with an individual object of a class. It means that all instances of the class share the same static data member. In other words, unlike regular (non-static members) data members, which have separate copies for each object, static data members are shared among all objects of the class.

```
class ClassName {  
    access_specifier:  
    static data_type data_member_name;  
};
```

- The **ClassName** and **data_member_name** refer to the names of the class and the data member we are creating.
- An **access specifier** indicates the visibility of the class member, which can be private, public or protected.
- The **static** keyword indicates that the respective member is of a static nature and the type of data it stores is given by **data_type**.

CHAPTER 4

Static Data Member

```
#include <iostream>
using namespace std;

class Mercedes {
public:
    // Declaration of static data member. Static members belong to the class itself, not to individual objects
    static int modelCount;

    // Constructor - automatically called when a Mercedes object is created
    Mercedes() {
        // Increment modelCount for each object created. modelCount is static, it keeps its value across all objects
        modelCount++;
    }
};

// Definition and initialization of static data member outside the class
// This allocates memory for the static variable
// Must be done exactly once in exactly one source file
int Mercedes::modelCount = 0;

int main() {
    // Creating Mercedes objects - each triggers the constructor
    Mercedes m1; // Creates first Mercedes object → modelCount becomes 1
    Mercedes m2; // Creates second Mercedes object → modelCount becomes 2
    Mercedes m3; // Creates third Mercedes object → modelCount becomes 3

    // Static members can be accessed even without creating any objects
    cout << "Total Mercedes models: " << Mercedes::modelCount << endl;

    return 0;
}
```

Context: Student count using Static Data Member

Problem Statement

- 1. Needed a Program that will output the total number of students(object) recorded*
- 2. Also demonstrate the difference with normal Data Member*

Access Specifiers & this Pointer

- private — accessible only inside the class's own member functions
- public — the class's external interface, accessible from anywhere
- protected — like private, but also accessible to derived classes
- this — hidden pointer to the current object; resolves name clashes, enables chaining

Real-life analogy: *private is the room safe, protected is a connecting door for family (derived classes), public is the front door.*

Constructors & Destructors

- Constructor — same name as class, no return type, runs automatically on creation
- Destructor (~ClassName) — runs automatically on destruction; releases resources
- Default constructor — takes no arguments
- Parameterized constructor — takes arguments to set initial state
- Copy constructor — `ClassName(const ClassName&);` deep copy needed if class holds pointers

Copy Constructor — Why Deep Copy Matters

```
class Name {
    char *text;
public:
    Name(const char *s){ text=new char[strlen(s)+1]; strcpy(text,s); }
    Name(const Name &src) {                // DEEP copy
        text = new char[strlen(src.text)+1]; // NEW memory
        strcpy(text, src.text);
    }
    ~Name() { delete[] text; }
};
// Without a hand-written copy ctor, two objects would share
// ONE pointer -> delete[] runs twice -> undefined behaviour
```

Static Members & Friend Functions

- static data member — ONE copy shared by every object of the class
- static member function — called via `ClassName::func()`; no 'this' pointer
- friend function — NOT a member, but explicitly granted access to private data
- friend class — every member function of that class gets the same access

Real-life analogy: A friend function is like giving your accountant a key to your private filing cabinet — trusted by name, not automatic.

Practice & Exam-style Questions

1. What is a constructor? Explain its types with examples.
2. What is a copy constructor? Why is it needed?
3. Explain static data members and static functions with an example.
4. What is a friend function? Write a program using one.

CHAPTER 5

Operator Overloading

Rules & restrictions · Unary/binary operators · Friend operators · Type conversion

Rules & Restrictions

- operator keyword redefines what +, -, ==, [], << mean for your class
- At least one operand must be a user-defined type
- Cannot overload: :: . .* ?: sizeof — and cannot change precedence/arity
- Member function: left operand = this. Friend function: both operands explicit

Overloading + and == (Complex numbers)

```
class Complex {
    double real, imag;
public:
    Complex(double r=0,double i=0){real=r;imag=i;}
    Complex operator+(const Complex &c) {
        return Complex(real+c.real, imag+c.imag);
    }
    bool operator==(const Complex &c) {
        return real==c.real && imag==c.imag;
    }
};
// Complex c = a + b;  ->  compiler rewrites as  a.operator+(b)
```

Friend Operators & Data Conversion

- `10 + complexObj` needs a `FRIEND operator+` (left operand isn't the class type)
- `operator<<` as a friend enables `cout << myObject`
- `Basic -> Object`: a single-argument constructor acts as implicit conversion
- `Object -> Basic`: `operator TargetType() const { ... }` (no return type, no params)
- `Object -> different Object`: a constructor in the destination class OR a conversion operator in the source class

Practice & Exam-style Questions

1. State the general rules for overloading operators.
2. Overload + to add two complex numbers (member function).
3. Explain unary vs binary operator overloading with examples.
4. Explain type conversion between basic types and class objects.

CHAPTER 6

Virtual Functions and Polymorphism

Early vs late binding · Virtual functions · Pure virtual & abstract classes · Virtual base classes

Early Binding vs Late Binding

- Early (static) binding — compiler decides which function to call, at compile time
- Late (dynamic) binding — decided at RUN TIME, based on the object's real type
- C++ implements late binding through virtual functions (conceptually via a vtable)

Real-life analogy: Early binding = a letter addressed to 'Room 5'. Late binding = a letter addressed to 'the current Manager'.

Virtual Functions in Action

```
class Shape {
    public: virtual void identify() {
        cout << "generic shape"; }
};

class Circle : public Shape {
    public: void identify() override {
        cout << "I am a Circle"; }
};

Shape *s = new Circle();
s->identify();    // calls Circle::identify — decided at RUN TIME
```

Pure Virtual Functions, Abstract Classes, Virtual Base Classes

- Pure virtual: `virtual double area() = 0;` (no body in base class)
- A class with ≥ 1 pure virtual function is ABSTRACT — cannot be instantiated
- Forces every derived class to implement that function
- Virtual base class (`virtual public Base`) — solves the 'diamond' duplicate-copy problem

Practice & Exam-style Questions

1. What is a virtual function? Why is it needed?
2. Differentiate compile-time vs run-time polymorphism.
3. What is an abstract class? Can it be instantiated?
4. Explain virtual base class with a diamond-inheritance example.

CHAPTER 7

Inheritance

Types of inheritance · Constructor/destructor order · Public/private/protected · Ambiguity · Containership

Types of Inheritance

- Single — one base, one derived
- Multiple — one derived, two or more base classes
- Multilevel — a chain: A -> B -> C
- Hierarchical — several derived classes share one base
- Hybrid — a combination (often produces the 'diamond' shape)

Construction Order & Access Modes

- Constructors run BASE first, then DERIVED
- Destructors run in REVERSE: DERIVED first, then BASE
- public inheritance — models a true 'is-a' relationship (most common)
- private inheritance — models 'is-implemented-in-terms-of', not 'is-a'
- Base-class private members are NEVER directly reachable from a derived class

Ambiguity in Multiple Inheritance

```
class Printer { public: void status(){cout<<"Printer ready";} };  
class Scanner { public: void status(){cout<<"Scanner ready";} };  
class AllInOne : public Printer, public Scanner {};  
  
AllInOne m;  
// m.status();           // ERROR: ambiguous!  
m.Printer::status();     // resolved explicitly  
m.Scanner::status();
```

Containership ('has-a'): a class holds an object of another class as a data member — an alternative to inheritance.

Practice & Exam-style Questions

1. Explain the types of inheritance with examples.
2. Differentiate public, private, and protected inheritance.
3. What is ambiguity in multiple inheritance? How is it resolved?
4. Differentiate inheritance and containership with an example.

CHAPTER 9

Templates and STL

Function templates · Class templates · Containers · Iterators · Algorithms

Function & Class Templates

- template <typename T> — one piece of code, compiler fills in the real type
- Function template: T maximum(T a, T b) works for int, double, char, ...
- Class template: Stack<T> can become Stack<int>, Stack<string>, ...
- Templates give reusability at the TYPE-SYSTEM level

***Real-life analogy:** A template is a cookie cutter — one shape, pressed into different doughs (types) to produce different cookies.*

STL: Containers, Iterators, Algorithms

- Containers — vector (dynamic array), list (linked list), map (key-value, sorted), set
- Iterators — generalised pointers; begin() / end() traverse any container uniformly
- Algorithms — sort(), find(), count(), for_each() operate on [begin, end) ranges
- Same sort() works on vector<int> AND vector<string> — that's the STL's power

Vector + Iterator + Algorithm Together

```
vector<int> marks = {56, 90, 45, 78, 88};  
sort(marks.begin(), marks.end());  
  
for (auto it = marks.begin(); it != marks.end(); ++it)  
    cout << *it << " ";  
// Output: 45 56 78 88 90
```

Practice & Exam-style Questions

1. Differentiate function template and class template.
2. What is STL? Explain its components.
3. Explain iterators and their role in STL.
4. Sort a vector of integers using STL sort().

CHAPTER 10

Namespaces

Scope resolution operator · The using keyword

Why Namespaces? Using Them

- A namespace gives a distinct scope to names, preventing naming collisions
- The entire C++ standard library lives inside namespace std
- Namespace::member — explicit, always safe, more to type
- using namespace X; — imports every name from X (convenient, but risks collisions)
- using X::specificName; — imports only ONE name (safer, targeted)

Real-life analogy: A namespace is like a surname: many 'Ram's in a village, but 'Ram Shrestha' vs 'Ram Thapa' never gets confused.

Declaring and Using Namespaces

```
namespace College { int studentCount = 250; }  
namespace Company { int studentCount = 40; } // no conflict  
  
cout << College::studentCount; // 250  
cout << Company::studentCount; // 40
```

Practice & Exam-style Questions

1. What is a namespace? Why is it used in C++?
2. Differentiate 'using namespace std;' vs 'std::cout'.
3. Write a program demonstrating user-defined namespaces.

CHAPTER 10

Exception Handling

try / catch / throw · Multiple catch blocks · User-defined exceptions

The Exception Handling Model

- Exception — an unusual/error condition arising at run time
- try — marks a block where an exception might occur
- throw — signals an exceptional condition, hands over an exception object
- catch — a handler that receives a matching thrown object
- Control jumps DIRECTLY from throw to a matching catch — skips remaining try code

Real-life analogy: A fire alarm: whoever notices the fire (throw) doesn't fight it themselves — the fire brigade (catch), positioned separately, handles it.

Basic try / throw / catch

```
double divide(double a, double b) {  
    if (b == 0)  
        throw runtime_error("Division by zero!");  
    return a / b;  
}  
  
try {  
    cout << divide(10, 0);  
} catch (runtime_error &e) {  
    cout << "Caught: " << e.what();  
}
```

Multiple catch Blocks & User-defined Exceptions

- Several catch handlers may follow one try, tried top to bottom
- `catch (...)` — catch-all safety net, usually placed last
- User-defined exceptions typically derive from `std::exception`
- Override `what()` (virtual in `std::exception`) to describe the error

Practice & Exam-style Questions

1. Explain try, catch, throw with a program.
2. Write a program to handle division-by-zero.
3. What is a user-defined exception? Write a program.
4. Differentiate exception handling from return-code error handling.

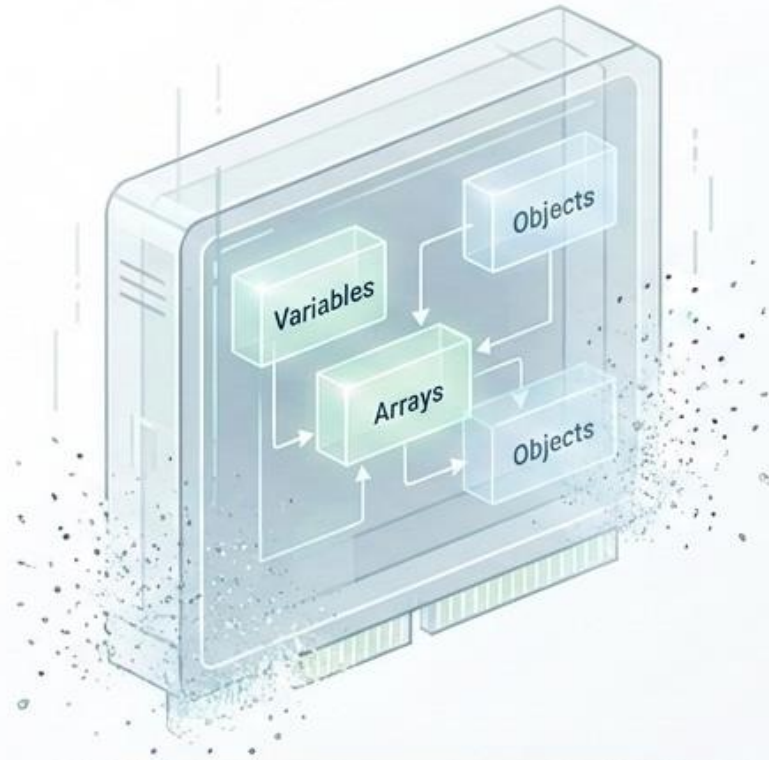
CHAPTER 11

File Handling

ofstream/ifstream/fstream · File modes · seekg/seekp/tellg/tellp · Reading & writing objects

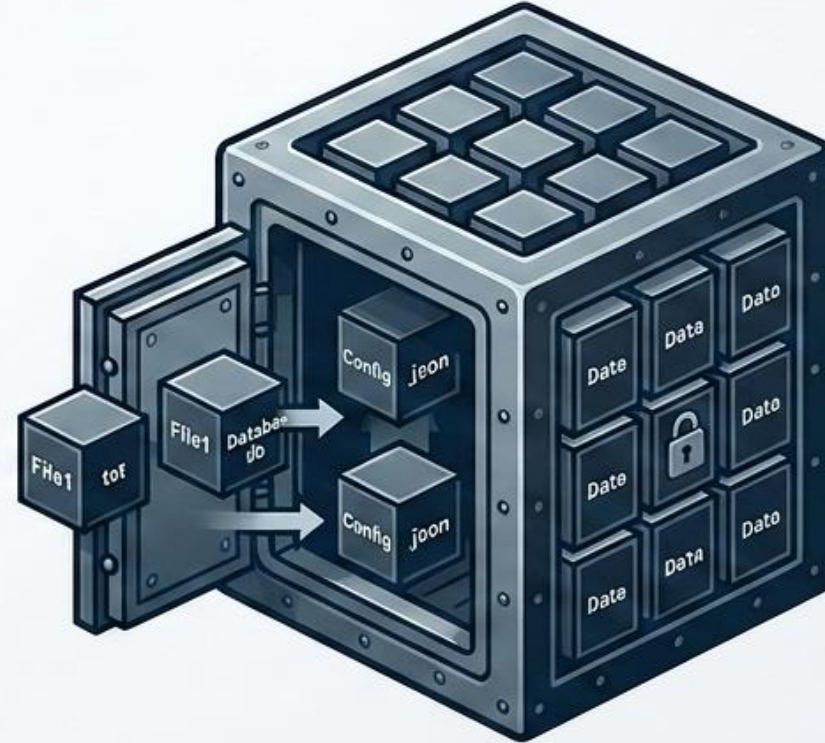
Temporary Execution vs. Persistent Storage

RAM (Volatile)



**Power Off / Program Exit
= Data Lost forever.**

Disk (Persistent)



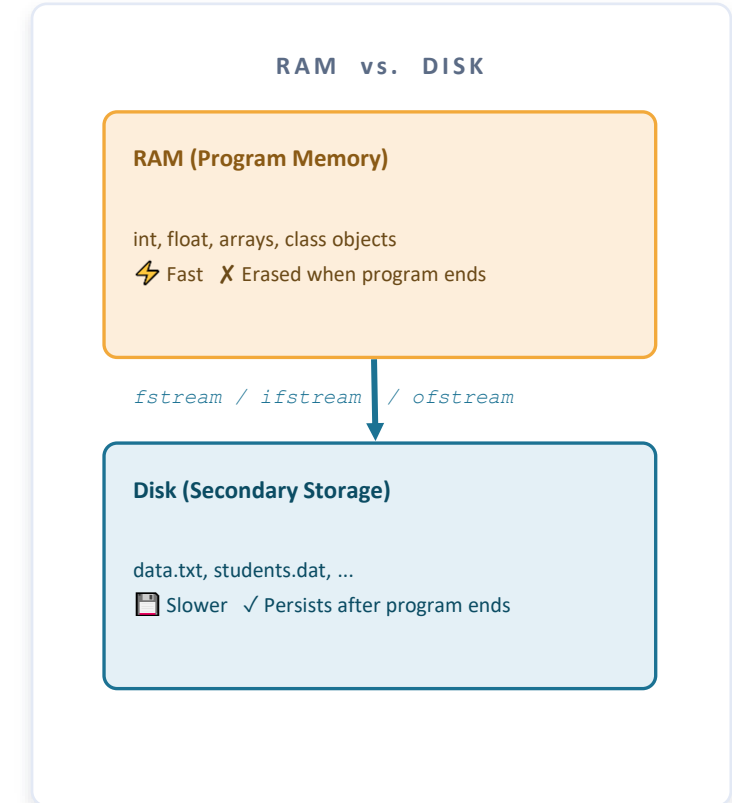
**Files survive termination.
Data is etched permanently.**

File handling bridges the gap between ephemeral memory and permanent storage, allowing programs to store data permanently, retrieve it across sessions, and handle volumes too large for system RAM.

File Handling

File handling is the mechanism by which a C++ program creates, opens, reads, writes, and closes files stored on secondary storage (disk), using stream classes from the `<fstream>` header so that data can persist beyond the program's lifetime.

- RAM (variables, arrays, objects) is volatile — its contents disappear the instant the program terminates.
- A disk file is persistent — it survives program exit, system reboot, and can be shared between programs.
- C++ treats every source of input and every destination of output — keyboard, screen, or file — uniformly as a stream of bytes.
- The header `<fstream>` supplies three classes: `ifstream` (input/read), `ofstream` (output/write), and `fstream` (both).



THE BIG ANALOGY

The Library Analogy

Keep this picture in your head for the entire lecture — every function we meet maps onto it.



YOU

The Program

You want to read or record information, but you never touch the shelves yourself.



The Librarian

The stream object (ifstream / ofstream / fstream) — the only one allowed to fetch, place or return books.



The Book

The disk file itself — pages of data sitting on a shelf (the disk) until someone requests it.



The Bookmark

The file pointer — marks exactly which page/word you'll read or write next.

Opening a file

The librarian walks to the shelf, retrieves the book, and lays it open on your desk — you may now use it.

Closing a file

The librarian saves any notes you made, then returns the book to the shelf. Skip this and notes can be lost.

seekg() / seekp()

You tell the librarian: “flip directly to page 50” — jump the bookmark anywhere instead of reading in order.

tellg() / tellp()

You ask: “what page is my bookmark on right now?” — the librarian reports the exact position.

get() / put()

Reading or writing one word at a time, in natural order, exactly where the bookmark currently sits.

read() / write() of an object

Photocopying an entire filled-in form in one motion — every field, byte-for-byte — instead of retyping it.

File Handling

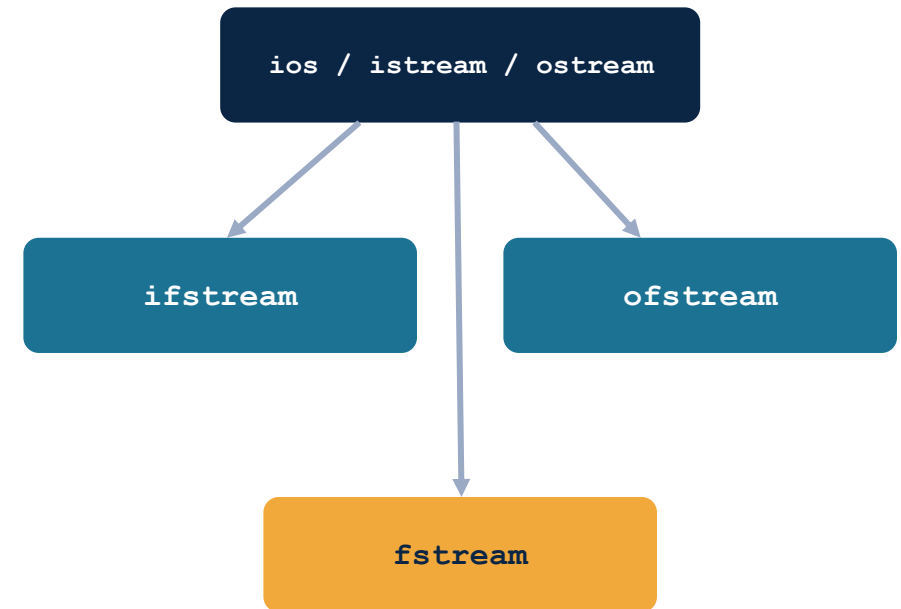
FOUNDATIONS

The fstream Family of Classes

```
#include <fstream>
```

- `ifstream` : “input file stream”. Derived for reading data FROM a file.
Default mode: `ios::in`.
- `ofstream` : “output file stream”. Derived for writing data TO a file.
Default mode: `ios::out` (truncates existing content).
- `fstream` : combines both; can read AND write the same file, useful for random-access/update operations.
- All three inherit stream-formatting behaviour from `ios`, so the familiar `<<` (insertion) and `>>` (extraction) operators work exactly as they do with `cout` and `cin`.

SIMPLIFIED CLASS HIERARCHY



read-only stream

write-only stream

read + write stream

File Handling

Opening and Closing a File — The Big Picture

- “Opening” a file means associating a stream object (ifstream/ofstream/fstream) with a physical file on disk, so that operations on the object act on that file.
- C++ gives two ways to open a file: via the constructor (at the moment the object is created) or via the open() member function (after the object already exists).
- A file MUST be closed with close() when you're done. Closing flushes any buffered data to disk and releases the operating-system file handle.
- Forgetting to close a file can lose buffered (not-yet-written) data, or leave the file locked/inaccessible to other programs.
- Always verify the file actually opened — test the stream object in a boolean context: `if (!file) { /* failed */ }`.

FILE LIFECYCLE

- 1 Declare stream object
- 2 Open (constructor or open())
- 3 Check it opened successfully
- 4 Read and/or write data
- 5 Close the file (close())

Streams and File Modes

- ofstream — write to a file | ifstream — read from a file | fstream — both
- Open via constructor: ofstream fout("data.txt");
- Open via .open(name, mode): ios::in, ios::out, ios::app, ios::ate, ios::trunc, ios::binary
- Always .close() a file when finished, to flush buffered data

Constructor Method

```
std::ofstream file("data.txt");
```

Implicitly opens upon object creation. Creates the file if it doesn't exist.

Open() Function Method

```
std::ofstream file;  
file.open("data.txt");
```

Explicit opening. Useful when reusing a single stream object for multiple different files.



```
file.close();
```

Critically **important**. Flushes any remaining buffer data to the disk and releases operating system locks.

Streams and File Modes

Opening a File Using the Constructor

- Pass the filename (and optionally, the mode) directly to the stream object's constructor at the moment it is declared.
- **Syntax:**
 - `ClassName streamObj("filename", mode);`
- Most concise option — ideal when you know the exact file you need right away and won't need to reuse the object for another file.
- If the mode is omitted, `ifstream` defaults to `ios::in` and `ofstream` defaults to `ios::out`.
- Limitation: you cannot re-open a different file with the same object this way — for that, use `open()` instead (next slide).

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Constructor opens the file directly -
    // no separate open() call is needed
    ofstream outFile("data.txt");    // mode: ios::out
    ifstream inFile("data.txt");     // mode: ios::in

    outFile << "Created using constructor!";
    outFile.close();

    // Always verify the constructor actually
    // succeeded in opening the file
    if (!inFile) {
        cout << "File could not be opened!";
    }

    inFile.close();
    return 0;
}
```

Streams and File Modes

Opening a File Using open()

- First declare the stream object with no file (default constructor), then bind it to a file later using open().
- **Syntax:**
 - `streamObj.open("filename", mode);`
- Lets you decide the filename at runtime (e.g., from user input) rather than hard-coding it at declaration.
- The SAME object can be reused: close() one file, then open() another — useful inside loops that process many files.
- The second argument (mode) can combine multiple flags with the bitwise-OR operator |, e.g. ios::in | ios::out | ios::app.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file;           // no file yet

    // Bind it to report.txt, allow reading,
    // writing, and always append new writes
    file.open("report.txt", ios::in | ios::out | ios::app);

    if (!file) { return 1; } // safety check

    file << "New entry logged." << endl;
    file.close();

    // Reuse the SAME object for another file
    file.open("log.txt", ios::out | ios::trunc);
    file << "Fresh log started.";
    file.close();

    return 0;
}
```

File Opening Modes(`ios::`)

- `ios::in` → Open for reading (input). Fails if the file does not exist (for `ifstream`).
- `ios::out` → Open for writing (output). Creates the file if absent; truncates (empties) it if present.
- `ios::app` → Append. Every write goes to the end of the file, no matter where the pointer is
- `ios::ate` → “At End”. Pointer starts at end-of-file on open, but writes may still occur anywhere.
- `ios::trunc` → Truncate. If the file exists, discard all its previous contents immediately.
- `ios::binary` → Open in binary mode — bytes are transferred exactly as stored, no text translation.

Flag	Mode	Behavior
<code>ios::in</code>	Read 🔍	Open for reading (Default for <code>ifstream</code>).
<code>ios::out</code>	Write ✍️	Open for writing. Deletes existing content upon opening (Default for <code>ofstream</code>).
<code>ios::app</code>	Append ➕	All output is safely appended to the absolute end of the file. Preserves previous data.
<code>ios::trunc</code>	Truncate 🗑️	Deletes all contents if the file already exists upon opening.
<code>ios::binary</code>	Binary 🖨️	Opens in raw machine byte mode instead of formatted text mode.



Modes can be combined using the bitwise OR operator (`|`). Example:
`ios::out | ios::app` safely opens a file to add new data without erasing previous contents.

Formatted I/O

- **Formatted I/O** means data is read or written according to its **data type and representation**.
- C++ converts data between its internal representation and a human-readable textual representation.
- The most familiar formatted I/O operators are:
 - ✓ << // insertion operator
 - ✓ >> // extraction operator

These above operators are used with:

- ✓ cout
- ✓ cin
- ✓ ofstream
- ✓ ifstream
- ✓ fstream

```
int main() {  
    ofstream file("student1.txt");  
  
    int age = 20;  
    double marks = 85.5;  
    string name = "Ram";  
  
    file << name << " " << age << " " << marks;  
  
    file.close();  
  
    return 0;  
}
```

Formatted I/O

```
class Student{
private:
    string name;
    int age;
    float marks;
public:
    // Function to assign student data
    void setData(string n, int a, float m){
        name = n;
        age = a;
        marks = m;
    }

    // Function to display student data
    void display(){
        cout << "Name   : " << name << endl;
        cout << "Age    : " << age << endl;
        cout << "Marks  : " << marks << endl;
    }

    // Formatted File Output
    void writeFormatted(){
        ofstream file("student2.txt");
        file << name << " " << age << " " << marks << endl;
        file.close();
    }

    // Formatted File Input
    void readFormatted(){
        ifstream file("student2.txt");
        file >> name >> age >> marks;
        file.close();
    }
};
```

```
int main(){
    Student s;

    // Store data in object
    s.setData("Ram", 20, 85.5);

    // -----
    // FORMATTED OUTPUT
    // -----
    s.writeFormatted();

    // Clear/change object data
    s.setData("", 0, 0);

    // -----
    // FORMATTED INPUT
    // -----
    s.readFormatted();

    cout << "Formatted Input:" << endl;
    s.display();

    return 0;
}
```

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    // Step 1: open file in write mode
    ofstream outFile;
    outFile.open("student.txt", ios::out);
    if (!outFile) {
        cout << "Error opening file!";
        return 1;
    }

    // Step 2: write data (text I/O)
    outFile << "Roll No: 101" << endl;
    outFile << "Name: Aarav Sharma" << endl;
    outFile << "Marks: 87.5" << endl;

    outFile.close();          // Step 3

    // Step 4: reopen the SAME file,
    // this time for reading
    ifstream inFile;
    inFile.open("student.txt", ios::in);
    if (!inFile) {
        cout << "Error reading file!";
        return 1;
    }

    // Step 5: read and display,
    // line by line
    string line;
    cout << "----- File Contents -----" << endl;
    while (getline(inFile, line)) {
        cout << line << endl;
    }

    inFile.close();          // Step 6
    return 0;
}

```

Step 1 : `outFile.open("student.txt", ios::out)` creates the file if it doesn't exist. Because mode is `ios::out`, any old content is discarded first.

Step 2: `outFile << "Roll No: 101" << endl;` sends characters through the stream's insertion operator — identical syntax to `cout`, different destination.

Step 3: Flushes the buffer to disk and releases the handle. Skipping this risks losing the last unflushed bytes and can block reopening.

Step 4: A NEW `ifstream` object opens the same filename with `ios::in`. Two different objects — one only ever writes, one only ever reads.

Step 5: Reads up to the next `'\n'` each call, returns the stream so the while condition becomes false automatically at end-of-file

CHAPTER 11

Positioning & Object I/O

- `tellg()`/`tellp()` : report current read/write position
- `seekg()`/`seekp()` (offset, direction) : move the position; `ios::beg` / `ios::cur` / `ios::end`
- `put()`/`get()` : single character I/O
- `write()`/`read()` : raw bytes; used to store WHOLE OBJECTS in binary mode
- Only classes with no raw pointers can be safely written/read as whole objects

CHAPTER 11

File Pointer

Think of a file as a sequence of characters/bytes:

```
student.txt
```

```
R a m      K u m a r
↑
File Pointer
```

The **file pointer** indicates the current position from which the next read or write operation will take place.

For example:

```
R a m K u m a r
↑
0
```

If the pointer moves forward:

```
R a m K u m a r
      ↑
      3
```

the next operation starts from position 3.

CHAPTER 11

File Pointer

- Every stream keeps two internal position markers
 - 'get' pointer (for reading)
 - 'put' pointer (for writing)
- For C++ file handling, there are four functions:

Function	Purpose
seekg()	Move the get/read pointer
seekp()	Move the put/write pointer
tellg()	Find the current get/read position
tellp()	Find the current put/write position

The easiest way to remember:

g → get → reading

p → put → writing

Therefore:

seekg() → move read pointer

seekp() → move write pointer

tellg() → tell read position

tellp() → tell write position

Positioning the File Pointer

tellg() - Find Read Position

- **tellg()** tells us the current position of the **get pointer**.

```
int main() {  
    ifstream file("student.txt");  
    cout << "Current position: " << file.tellg();  
    file.close();  
}
```

tellp() - Find Write Position

- **tellp()** tells us the current position of the **put pointer**

```
int main() {  
    ofstream file("data.txt");  
    cout << "Position: " << file.tellp();  
    file.close();  
}
```

CHAPTER 11

Positioning the File Pointer

tellg() and tellp()

- tellg() returns the CURRENT position of the GET (read) pointer, as an integer byte offset from the start of the file.
- tellp() returns the CURRENT position of the PUT (write) pointer, in the same way.
- Both take no arguments and return a value of type streampos (safely convertible to a plain integer for display or arithmetic).
- **Typical uses:**
 - measuring the total size of a file — seekg(0, ios::end) then tellg()
 - remembering a position to seekg() back to later
 - confirming how many bytes were just written, via tellp()

```
int main() {
    fstream file;

    //open file sample_TP.txt in and Write mode
    file.open("sample_TP.txt",ios::out);

    if(!file){
        cout<<"Error in creating file!!!";
        return 0;
    }

    //write A to Z
    file<<"ABCDEFGHIJKLMNOPQRSTUVWXYZ";

    //print the position
    cout<<"Current position is: "<<file.tellp()<<endl;
    file.close();

    //Again open file in read mode
    file.open("sample_TP.txt",ios::in);
    if(!file){
        cout<<"Error in opening file!!!";
        return 0;
    }
    cout<<"After opening file position is: "<<file.tellg()<<endl;

    //read characters untill end of file is not found
    char ch;
    while(!file.eof()){
        cout<<"At position : "<<file.tellg();    //current position
        file>>ch;    //read character from file
        cout<<" Character \"<<ch<<"\"<<endl;
    }

    //close the file
    file.close();
    return 0;
}
```

Positioning the File Pointer

seekg() - Move Read Position

seekg() in C++ is a function that helps to get an arbitrary file position in the istream library. It is present in the fstream header file and is also a part of file handling, used to extract from the input stream by placing the pointer to the next character that is desired.

Suppose:

ABCDEFGH
01234567

We want to start reading from position 4.

```
file.seekg(4);
```

A B C D E F G H
 ↑
 4

The next character read will be: E

```
int main() {  
    ifstream file("data.txt");  
    file.seekg(4);  
    char ch;  
    file.get(ch);  
  
    cout << "Character: " << ch;  
    file.close();  
    return 0;  
}
```

If **data.txt** contains:

ABCDEFGH

Output:

E

CHAPTER 11

Positioning the File Pointer

seekp() - Move Write Position

`seekp()` is used with an output file stream to move the **put/write position**.

Suppose:

ABCDEFGH
01234567

We want to start reading from position 4.

```
ofstream file("data.txt");  
file.seekp(4);  
file.put('X');
```

Before:

A B C D E F G H
 ↑
 4

After: A B C D X F G H

The character at position **4** has been replaced.

```
int main() {  
    fstream file("data.txt", ios::in | ios::out);  
    file.seekp(4);  
    file.put('X');  
    file.close();  
    return 0;  
}
```

If **data.txt** contains:

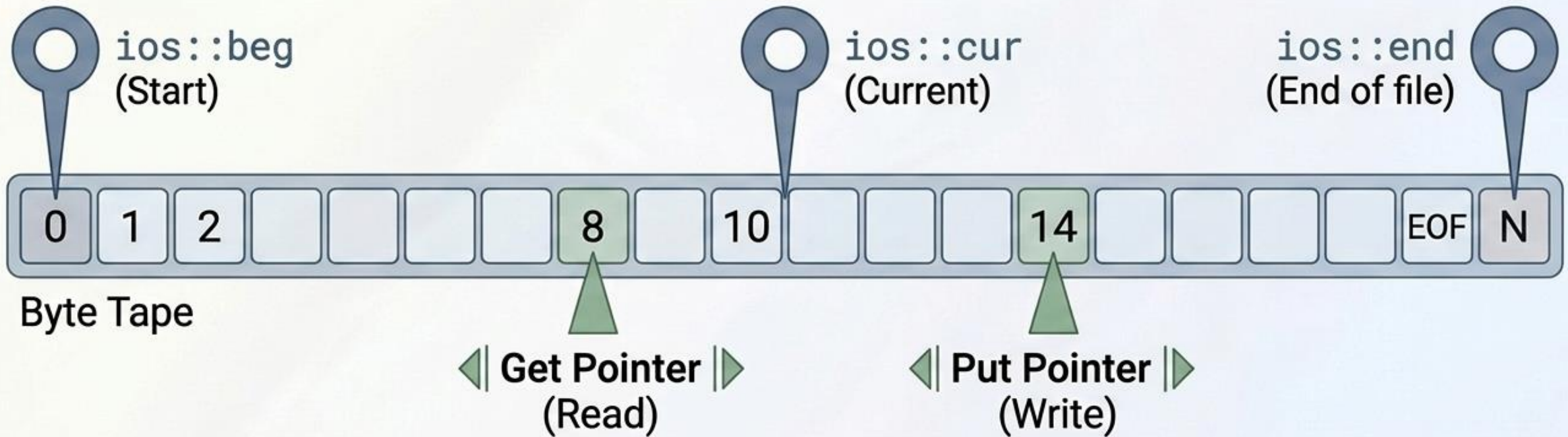
ABCDEFGH

After Execution:

ABCDXFGH

The Anatomy of Random Access

Sequential reading is like a VHS tape. Random Access is like a DVD:
jump instantly to any **exact byte offset** in the file.



C++ maintains two independent position pointers. You can command these pointers to jump to any specific byte address in the entire file.

Positioning the File Pointer

seekg() and seekp()

- Every open file keeps two internal position markers: a GET pointer (for reading) and a PUT pointer (for writing).
- `seekg(offset, direction)` — moves the GET pointer. Think: “seek for get.”
- `seekp(offset, direction)` — moves the PUT pointer. Think: “seek for put.”
- direction is one of three anchors:
 - `ios::beg` — offset counted from the beginning of the file
 - `ios::cur` — offset counted from the CURRENT position
 - `ios::end` — offset counted backwards from the end of file
- offset can be negative when combined with `ios::cur` or `ios::end`, to move backwards.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file;
    file.open("alphabet.txt", ios::in | ios::out);

    file.seekg(4, ios::beg);
    // -> jump the READ pointer to byte index 4
    //      (the 5th character, 0-indexed)

    file.seekp(0, ios::beg);
    // -> jump the WRITE pointer back to the start

    file.seekg(-1, ios::cur);
    // -> step the READ pointer 1 byte backwards
    //      from wherever it currently is

    file.seekg(-2, ios::end);
    // -> position READ pointer 2 bytes before EOF

}
```

Unformatted I/O

- **Unformatted I/O** transfers characters or blocks of data **without performing the usual type-based formatting/conversion**.

Common unformatted functions include:

- ✓ get()
- ✓ getline()
- ✓ put()
- ✓ read()
- ✓ write()
- ✓ peek()
- ✓ ignore()

```
int main() {  
    ifstream file("student1.txt");  
  
    int age = 20;  
    double marks = 85.5;  
    string name = "Ram";  
  
    file << name << " " << age << " " << marks;  
  
    file.close();  
  
    return 0;  
}
```


CHAPTER 11

get() - Find One character

```
int main() {  
    ifstream file("student.txt");  
    char ch;  
    while (file.get(ch)) { cout << ch; }  
    file.close();  
}
```

put() - Write One character

```
int main() {  
    ifstream file("output.txt");  
  
    file.put('A');  
    file.put('B');  
  
    file.close();  
}
```

getline() - Read an Entire Line

```
int main() {  
    ifstream file("student.txt");  
    string line;  
    getline(file, line);  
    cout << line;  
  
    file.close();  
}
```

CHAPTER 11

read() and write() - Block I/O

These are especially important when dealing with **binary files**.

write() : Writes a block of bytes.

```
file.write((char*)&student, sizeof(student));
```

read() : Reads a block of bytes.

```
file.read((char*)&student, sizeof(student));
```

```
class Student{
public:
    int id;
    char name[20];
    float marks;
};

int main()
{
    Student s = {101, "Ram", 85.5};

    ofstream file("student.dat", ios::binary);

    file.write((char*)&s, sizeof(s));

    file.close();

    return 0;
}
```

CHAPTER 11

Writing/Reading a Whole Object (Binary)

```
#include <iostream>
#include <fstream>
using namespace std;

class Student {
    int rollNo;
    char name[30];
    float marks;
public:
    void getData() {
        cout << "Roll, Name, Marks: ";
        cin >> rollNo >> name >> marks;
    }
    void showData() {
        cout << rollNo << "\t" << name << "\t" << marks << endl;
    }
};

int main() {
    Student s1;
    s1.getData();

    ofstream outFile("students.dat",ios::out | ios::binary);
    outFile.write(reinterpret_cast<char*>(&s1), sizeof(Student));
    outFile.close();

    Student s2;    // empty, untouched object
    ifstream inFile("students.dat",ios::in | ios::binary);
    inFile.read(reinterpret_cast<char*>(&s2),sizeof(Student));
    inFile.close();

    cout << "\nRead back from file:\n";
    s2.showData();
    return 0;
}
```

Practice & Exam-style Questions

1. Explain the different file opening modes in C++.
2. Explain seekg(), seekp(), tellg(), tellp() with an example.
3. How are class objects read from/written to files?
4. Differentiate ios::app and ios::ate.



Probable Exam Question

Q1: Write a C++ program to read student records from a file, search by roll number, and display the result. Explain file pointer manipulation functions with examples.
[PU Exam 2022]

Thank You